SĂPTĂMÂNA 4

Teoria cozilor și performanța sistemelor de calcul

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Unde am rămas

- ✓ Procesul **Poisson**: sosiri independente; timpii dintre sosiri sunt **exponențiali** (S2)
- ✓ Lanțuri Markov: stări, tranziții, **distribuția staționară** $\pi = \pi P$ (S3)
- ✓ Echilibrul fluxurilor: „cât intră într-o stare, atât iese” ($\pi_1 a = \pi_2 b$)
- ✓ Azi: o coadă este un lanț Markov în care **starea = numărul de clienți**

La finalul săptămânii veți putea…

1. să descrieți un sistem de servire în **notația Kendall** (A/B/c/K)
2. să calculați **indicii de performanță**: ρ, L, Lq, W, Wq, throughput
3. să aplicați **legea lui Little** și legile operaționale (gâtul de sticlă)
4. să folosiți formulele **M/M/1, M/M/c, M/M/1/K** și Erlang
5. să explicați de ce **latența explodează** când utilizarea se apropie de 100%
6. să analizați o **rețea de cozi** (aplicație web, cloud, servirea LLM)

Ora 1

Sisteme de servire și legea lui Little

1. Clienți, stații, cozi, discipline; notația Kendall
2. Sosiri Poisson și indicii de performanță
3. Legea lui Little și legile operaționale
4. Variabilitatea creează așteptare

Cozile sunt peste tot

- Oameni: ghișee, casele din supermarket, urgențele unui spital
- Calculatoare: procese care așteaptă **CPU**, cereri de **I/O** la disc
- Rețele: pachete în **bufferele** unui router
- Aplicații: cereri HTTP, conexiuni la **baza de date**, mesaje într-o coadă (Kafka)
- AI: cereri care așteaptă un **GPU** pentru inferență

? Întrebarea centrală

Cât timp așteaptă un client și **câte resurse** ne trebuie ca așteptarea să fie acceptabilă?

Motivația: încărcarea costă timp (M/M/1)

50% → ×2

server ocupat jumătate din timp: timpul de răspuns = **2 ×** timpul de servire

90% → ×10

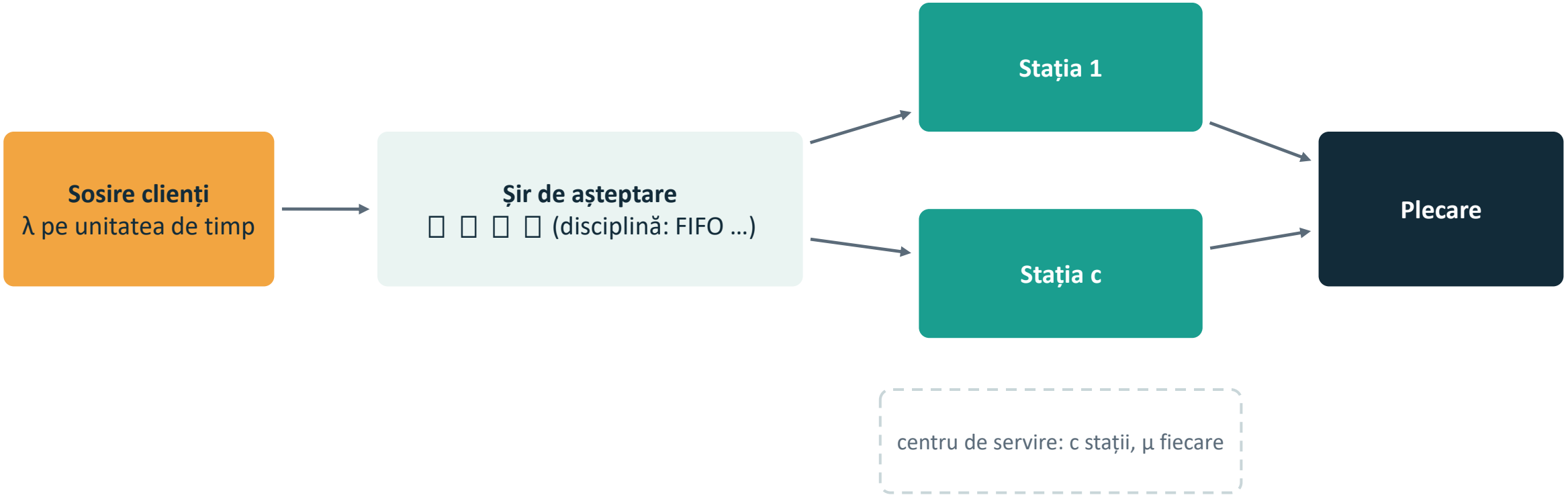
ocupat 90% din timp: **10 ×** timpul de servire

99% → ×100

ocupat 99% din timp: **100 ×** timpul de servire

- Relația nu este liniară: ultimele procente de utilizare sunt cele mai scumpe
- Vom deduce formula $W = 1/(\mu - \lambda)$ în ora a doua

Un sistem de servire (Holban, cap. 3)



Clienții sosesc, așteaptă dacă stațiile sunt ocupate, sunt serviți și pleacă

Vocabularul

Sistem de servire

Clienții emit cereri de servire; o **stație de servire** servește un singur client la un moment dat; clienții care găsesc stațiile ocupate formează un **șir de așteptare**, ordonat după o **disciplină**.

Holban, cap. 3.1

- Client = orice cerere: persoană, pachet, job, interogare SQL, prompt
- Stație = orice resursă: casier, CPU, fir de execuție, conexiune, GPU
- Timpul total în sistem = așteptare + servire

Scurt istoric

Erlang (Copenhaga): câte linii telefonice îi trebuie unei centrale?

1909–17

1930–32

Pollaczek, Khinchine: formula pentru servire oarecare (M/G/1)

Kendall: notația A/B/c; lanțuri Markov incluse

1953

1957–61

Jackson: rețele de cozi;
Little: L = λW

Kleinrock: cozi pentru rețele de calculatoare (ARPANET)

1960–70

2013 →

„The Tail at Scale”; servirea LLM (Orca 2022, vLLM 2023)

Notația Kendall

$$A / B / c / K / N / D$$

A / B	distribuția timpilor între sosiri / a timpilor de servire: **M** (exponențial), **D** (constant), **G** (oarecare)
c	numărul de stații (servere) în paralel
$K / N / D$	capacitatea sistemului, mărimea populației, disciplina – lipsesc dacă sunt ∞, ∞, FIFO

M vine de la „Markov” (fără memorie). Exemplu: **M/M/2/10** = sosiri Poisson, servire exponențială, 2 servere, cel mult 10 clienți în sistem.

Modele pe care le vom folosi

Model	Ce înseamnă	Unde apare
M/M/1	un server, sosiri Poisson, servire exponențială	modelul de bază; un CPU, un disc
M/M/c	c servere și o coadă comună	call center, fire de execuție, GPU-uri
M/M/1/K	capacitate finită: clienții în plus sunt refuzați	buffere de rețea, cozi limitate (HTTP 429)
M/M/c/c	fără coadă: pierdere dacă toate sunt ocupate	linii telefonice, licențe (Erlang B)
M/D/1, M/G/1	servire constantă / oarecare	pachete de lungime fixă, cereri variate
G/G/c	totul oarecare	aproape orice sistem real → **simulare**

Exercițiu de notație: benzinăria (Holban, studiu de caz 1)

Element	Descriere
Clienți	mașini care vin să alimenteze
Sosiri	exponențiale, în medie la fiecare 240 u.t. → **M**
Servire	normală, media 96 u.t., abaterea 32 u.t. → **G**
Stații	2 pompe, o coadă comună → **c = 2**
Notația Kendall	**M/G/2**; ρ = λ·s / c = (96/240) / 2 = **0,2**

Holban modelează sistemul ca graf de activități: sosire → așteptare → alimentare → plecare

Discipline de servire

1 FIFO / FCFS

Primul venit, primul servit. Cozile din magazine, multe servere.

2 LIFO

Ultimul venit, primul servit. Stive, unele cozi de joburi.

3 Priorități

Clase de clienți: urgențe, trafic de control în rețele.

4 Round-robin / PS

Fiecare primește o cuantă de timp: planificatorul unui sistem de operare.

5 SJF / SRPT

Cel mai scurt job întâi: reduce timpul mediu, dar joburile lungi pot aștepta mult.

6 Aleator

Clientul servit este ales la întâmplare (unele protocoale de rețea).

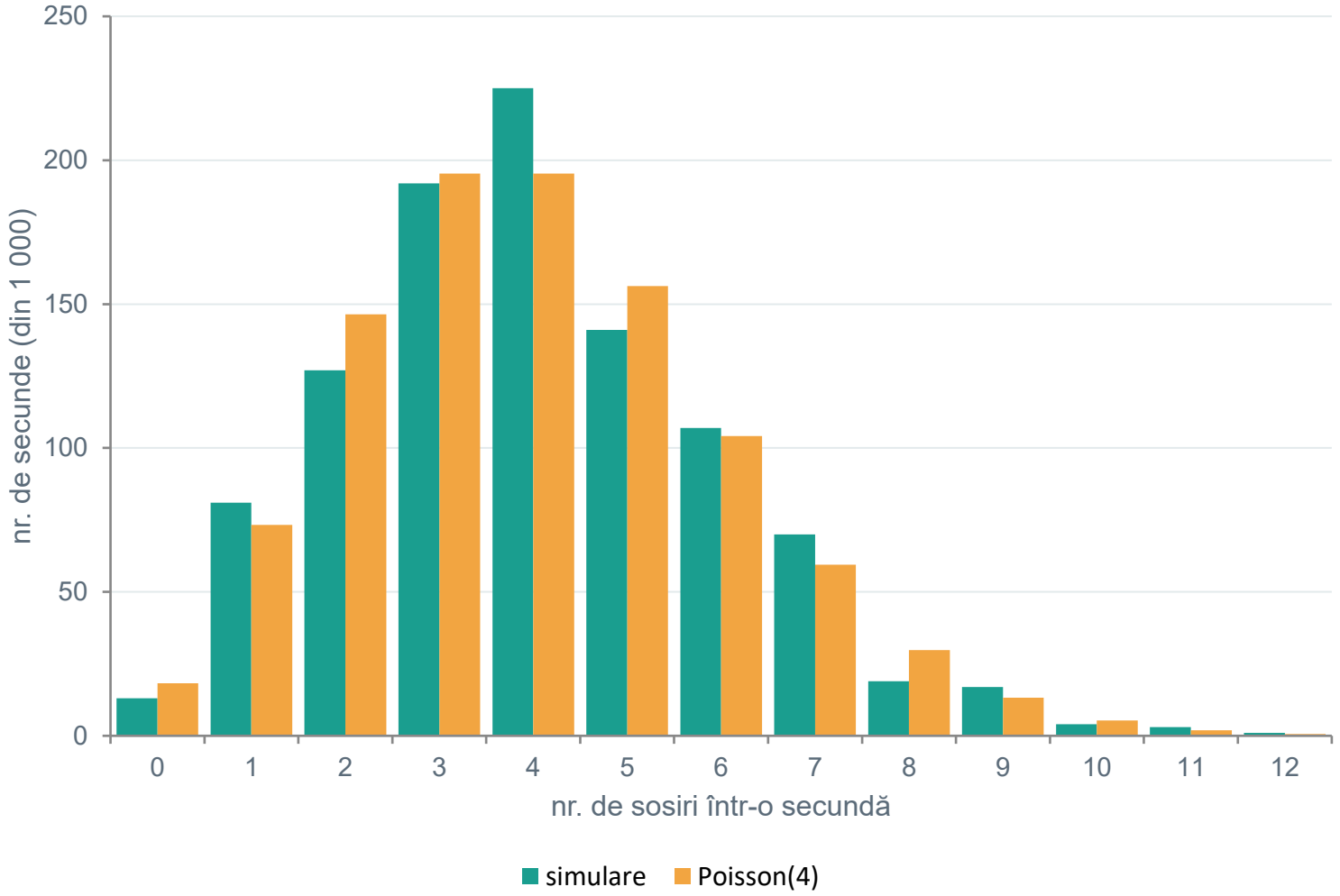
Sosirile: procesul Poisson (Holban, cap. 3.3)

$$P(k \text{ sosiri în } T) = e^{-\lambda T} (\lambda T)^k / k!$$

ipoteze	sosiri independente, rată constantă λ, fără sosiri simultane
între sosiri	timpi **exponențiali** cu media 1/λ – fără memorie (Holban, 3.4–3.5)
media = var.	numărul de sosiri într-un interval T are media și dispersia λT

Profesorul Holban deduce formula din binomială: împărțim T în intervale mici h, fiecare cu o sosire cu probabilitatea λh, și trecem la limită.

Sosiri simulate: 4 cereri/s, numărate secundă cu secundă



- 4 038 de sosiri în 1 000 s
- Media pe secundă: **4,04**, dispersia: **3,92**
- Media ≈ dispersia: semnătura procesului Poisson
- Uneori 0 sosiri, alteori 10: sosirile vin în „ciorchini”

De ce Poisson? Și când nu?

- Multe surse **independente** și mici (utilizatori) → suprapunerea lor e aproape Poisson
- Lipsa memoriei face calculele simple: starea = numărul de clienți
- **Nu** e Poisson: rafale (bursts), sosiri sincronizate (cron la ora fixă), retry-uri
- **Nu** e constant: trafic zi/noapte → λ(t) variabil (S6, laboratorul „time slots”)
- Verificați pe date: media ≈ dispersia? timpii dintre sosiri exponențiali?

Regulă practică

Poisson e un punct de plecare bun pentru sosiri; pentru **servire**, exponențiala e mult mai rar realistă.

Indicii de performanță (Holban, cap. 3.2)

$$\lambda, \quad s = 1/\mu, \quad \rho = \lambda / (c \cdot \mu), \quad X = min(\lambda, c \cdot \mu)$$

λ, μ	rata de sosire; rata de servire a unei stații (μ = 1 / timpul mediu de servire s)
ρ	**gradul de utilizare**: fracțiunea din timp în care stațiile sunt ocupate
X	**capacitatea de trecere** (throughput): clienți serviți pe unitatea de timp

Plus: **Wq** (așteptarea), **W** = Wq + s (timpul de răspuns), **Lq** (lungimea cozii), **L** (clienți în sistem).

Exemplu lucrat (Holban): 4 clienți în 10 u.t.

Mărime	Calcul	Valoare
Rata de sosire λ	4 clienți / 10 u.t.	**0,4** cereri/u.t.
Timpul mediu de servire s	(0,3 + 1,2 + 0,7 + 1) / 4	**0,8** u.t.
Rata de servire μ	1 / 0,8	**1,25** cereri/u.t.
Gradul de utilizare ρ	3,2 u.t. ocupat / 10 u.t. = λ/μ	**0,32** (32%)
Capacitatea de trecere X	min(λ, μ)	**0,4** cereri/u.t.

Utilizarea se poate calcula direct (timp ocupat / timp total) sau ca λ·s: ambele dau 0,32

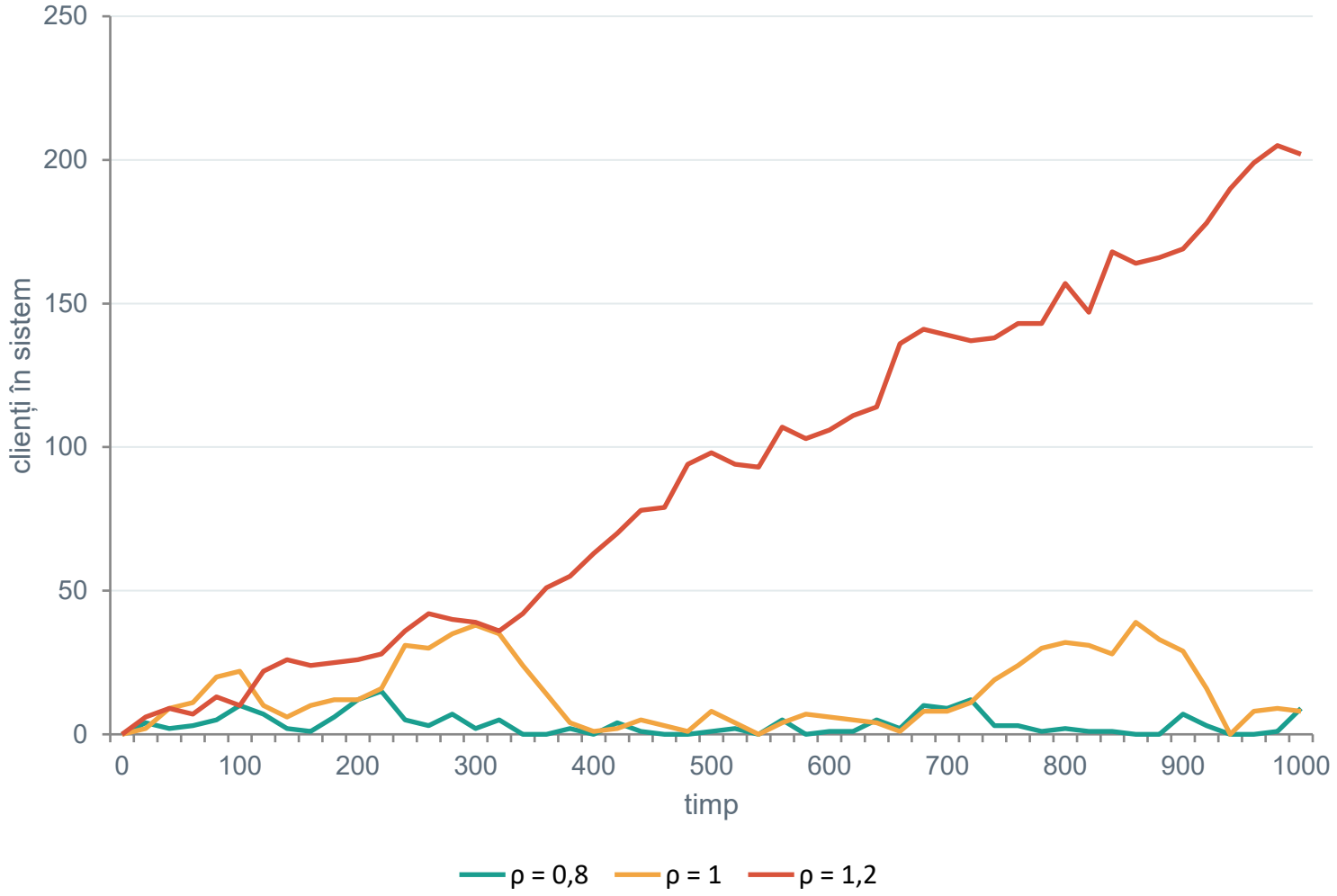
Condiția de stabilitate

$$\rho = \lambda / (c \cdot \mu) < 1$$

$\rho < 1$	serverele pot ține pasul; coada are o distribuție staționară
$\rho = 1$	la limită: coada nu converge, fluctuează fără margine
$\rho > 1$	clienții sosesc mai repede decât pot fi serviți: coada crește **liniar** în timp

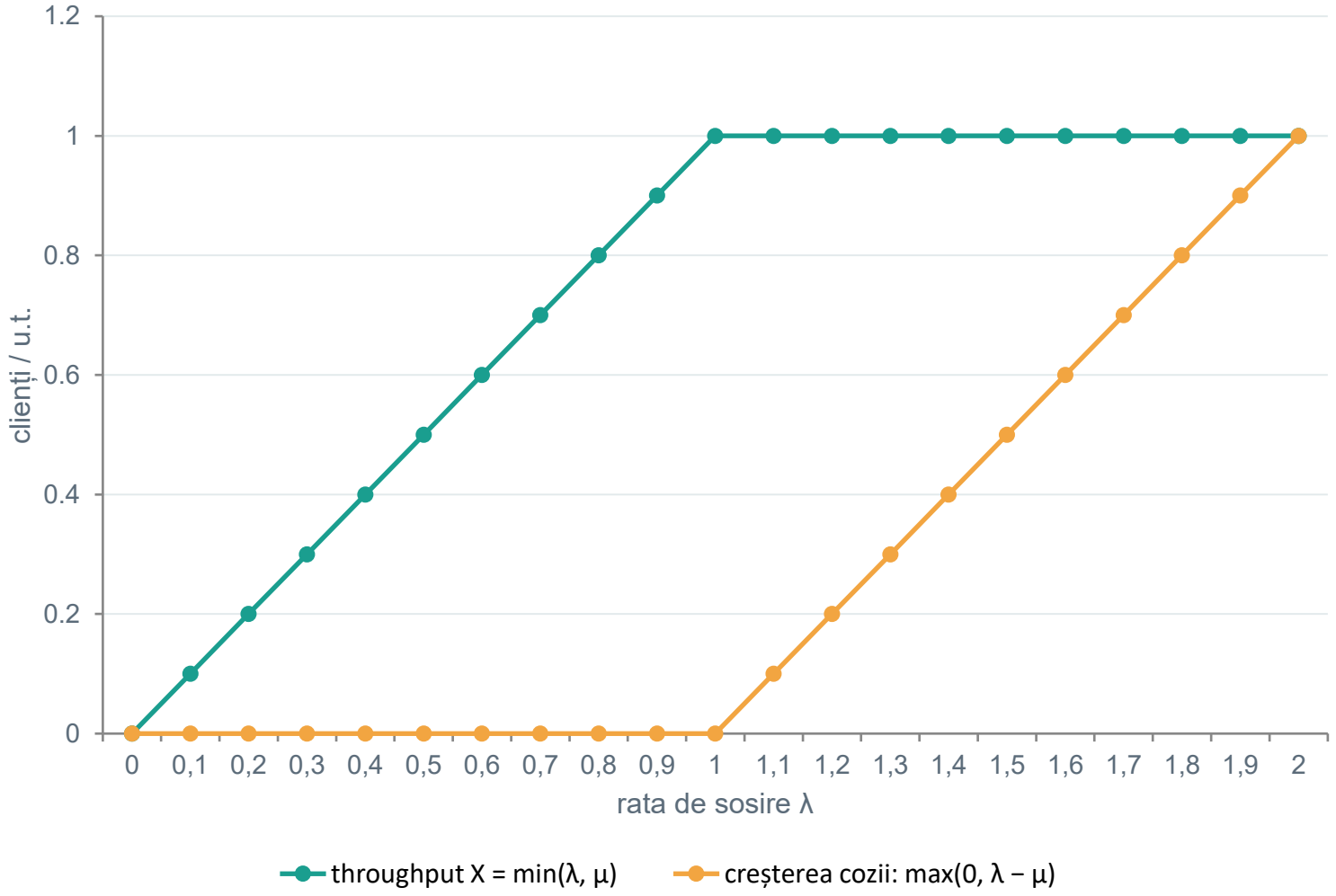
Capacitatea medie ajunge nu este suficientă: la ρ = 0,95 sistemul e stabil, dar așteptările sunt deja uriașe.

Numărul de clienți în timp (M/M/1, μ = 1)



- ρ = 0,8: fluctuează, dar revine mereu
- ρ = 1: se plimbă fără margine (o plimbare aleatoare!)
- ρ = 1,2: crește liniar, ≈ (λ − μ)·t = 0,2·t
- Aceeași sămânță, trei destine diferite

Throughput și coadă când crește încărcarea (μ = 1)



- Până la λ = μ, tot ce intră iese: X = λ
- Peste μ, throughput-ul se **saturează**
- Diferența λ − μ se acumulează în coadă
- Throughput-ul **nu** arată congestia: latența o arată

Două perspective asupra aceluiași sistem

Operatorul (proprietarul resursei)

- vrea **utilizare** mare: resursele plătite să lucreze
- măsoară throughput, ρ, costul pe cerere
- tentație: „serverul e la 95%, perfect!”

vs

Utilizatorul

- vrea **latență** mică și predictibilă
- simte W și mai ales **p95 / p99**
- la 95% utilizare, latența e ≈ 20 × timpul de servire

Timpi și lungimi

Patru mărimi de bază

Wq = timpul mediu de așteptare în coadă; **W = Wq + s** = timpul mediu de răspuns (în sistem); **Lq** = numărul mediu de clienți în coadă; **L = Lq + (clienți în servire)** = numărul mediu în sistem.

- Timpii se măsoară per client, lungimile se mediază în timp
- Pentru utilizatori contează W (și percentilele lui), pentru resurse contează ρ
- Q(t) = max(0, N(t) − c): lungimea cozii la momentul t (Holban)

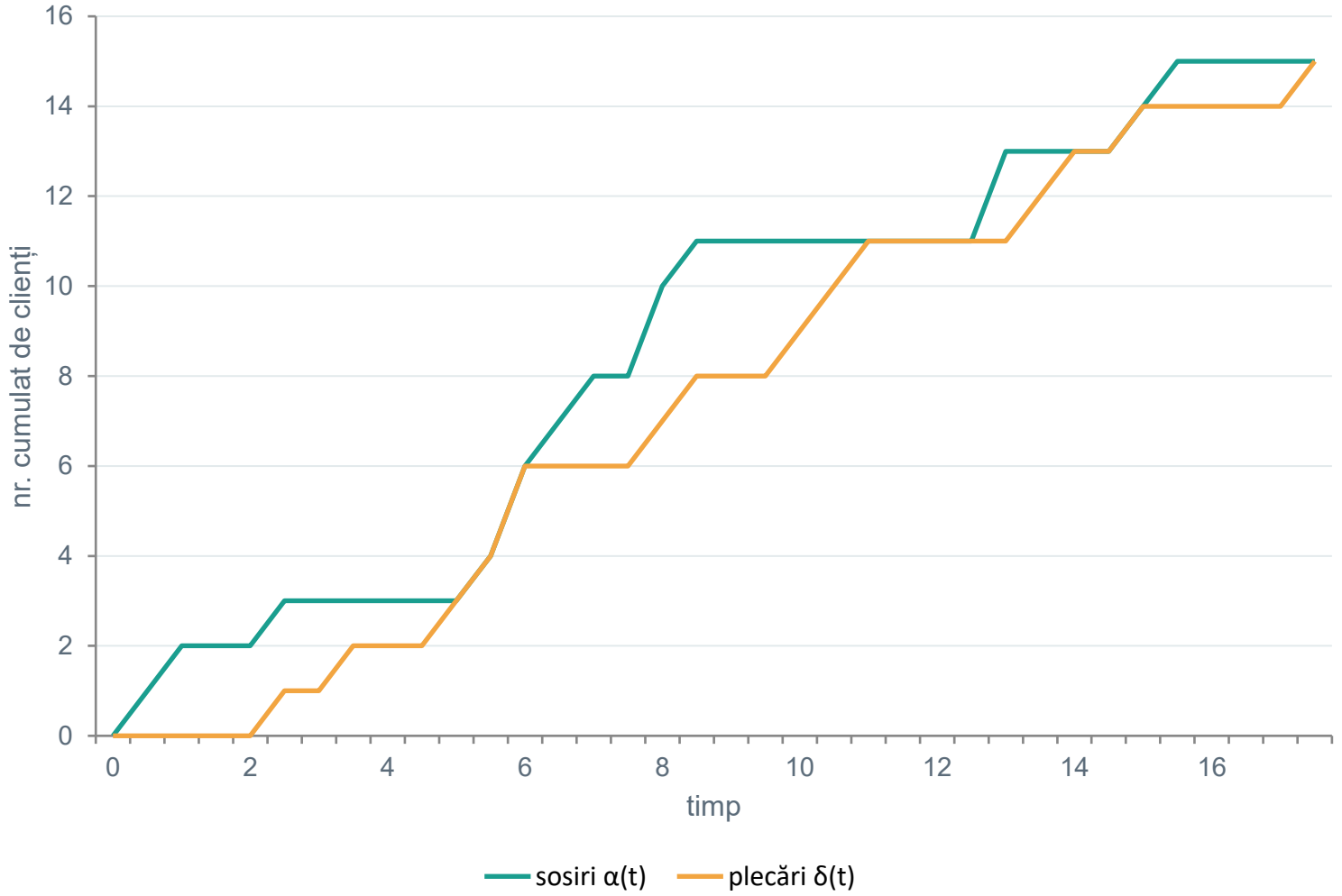
Legea lui Little (1961)

$$L = \lambda \cdot W$$

L	numărul mediu de clienți în sistem
λ	rata medie de sosire (= rata de plecare, în regim staționar)
W	timpul mediu petrecut în sistem

Nu presupune **nimic** despre distribuții, disciplină sau numărul de servere. Funcționează pentru orice „cutie” cu intrări și ieșiri.

Demonstrația intuitivă: aria dintre două curbe



- Distanța **verticală** α(t) − δ(t) = clienți în sistem la momentul t
- Distanța **orizontală** = timpul petrecut de un client
- Aria dintre curbe = Σ timpi = ∫ N(t) dt
- Împărțind aria la t, respectiv la numărul de clienți → **L = λW**

Little se aplică oricărei „cutii”

- **Coada**: $L_q = \lambda \cdot W_q$
- **Serverele**: clienți în servire = $\lambda \cdot s = c \cdot \rho$ (legea utilizării)
- **Sistemul**: $L = \lambda \cdot W = L_q + \lambda \cdot s$
- **Un întreg centru de date**: cereri în curs = rata cererilor × latența medie
- Cunoaștem două mărimi → o aflăm pe a treia, **fără** niciun model

C **Exemplu Cambridge**

Un disc servește o cerere în 10 ms, iar cererile vin cu 100/s: în servire se află în medie 100 × 0,01 = **1** cerere.

Little în trei exemple

10

server web: **200** cereri/s × **50 ms** → 10 cereri în curs de procesare

6

cafenea: **30** clienți/oră × **12 min** → 6 clienți înăuntru, în medie

160

serviciu LLM: **20** cereri/s × **8 s** → 160 de conversații generate simultan

- Folosiți unități consistente: cereri/s cu secunde, clienți/oră cu ore
- Ultimul exemplu dimensionează memoria GPU (ora 3)

Verificare

? Un API primește în medie **50 de cereri/s**, iar timpul mediu de răspuns este **200 ms**. Câte cereri sunt în curs de procesare, în medie?

A 0,25

B 4

C 10

D 250

Little verificat prin simulare (G/G/1)

Mărime măsurată (100 000 de clienți)	Valoare
Rata de sosire $\hat{\lambda}$ (sosiri exponențiale)	0,702 clienți/u.t.
Timpul de servire (lognormal, coadă grea)	media 1,01 u.t.
Timpul mediu în sistem $\hat{W}$	4,438 u.t.
$\hat{\lambda} \cdot \hat{W}$	3,114
$\hat{L}$ observat (20 000 de momente aleatoare)	3,115

Servirea nu este exponențială, disciplina e FIFO: legea lui Little rămâne valabilă

Legile operaționale

$$U = X \cdot S \qquad X_i = V_i \cdot X \qquad D_i = V_i \cdot S_i$$

$U = X \cdot S$	**legea utilizării**: utilizare = throughput × timpul de servire
$X_i = V_i \cdot X$	**fluxul forțat**: o cerere vizitează resursa i de V_i ori
D_i	**cererea de servire** a resursei i: timpul total cerut de o cerere

Mărimi măsurabile direct (Jain; Lazowska et al.), fără ipoteze despre distribuții – ca legea lui Little.

Exemplu: unde e gâtul de sticlă?

Resursă	Vizite V_i	Servire S_i	Cerere $D_i = V_i \cdot S_i$
CPU	1	10 ms	10 ms
Disc	3	8 ms	24 ms
Baza de date	2	15 ms	30 ms
Total			**64 ms**

Gâtul de sticlă = resursa cu D_i maxim (baza de date, 30 ms): $X \leq 1/Dmax = 33{,}3$ cereri/s; $W \geq \Sigma D_i = 64$ ms

Verificare

? O cerere vizitează un disc de **4** ori, iar fiecare vizită durează **10 ms**. Care este throughput-ul **maxim** pe care îl permite discul?

A 4 cereri/s

B 25 de cereri/s

C 100 de cereri/s

D 400 de cereri/s

Unde se pierde timpul într-o cerere web?

1 Rețea

Latența de transmisie și cozile din routere.

2 Load balancer

Coada de conexiuni în fața serverelor.

3 Fire de execuție

Cereri care așteaptă un worker liber (thread pool).

4 Conexiuni la BD

Pool limitat: cererile așteaptă o conexiune.

5 Disc / BD

Interogări care așteaptă I/O sau blocări (locks).

6 Servicii externe

Apeluri către alte API-uri, fiecare cu coada lui.

O coadă în 10 linii: recurența lui Lindley

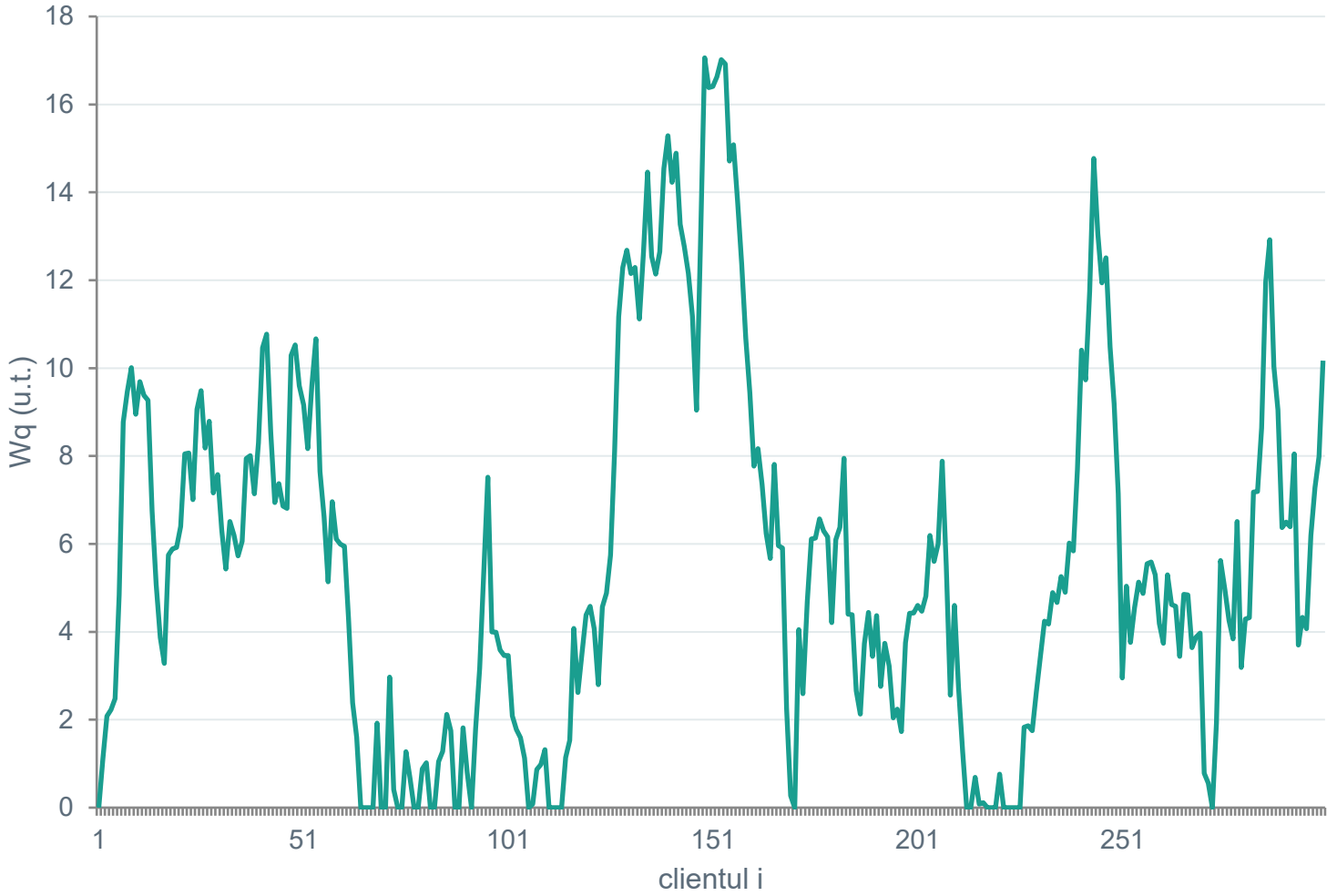
```
import numpy as np
rng = np.random.default_rng(5)
n = 200_000
A = rng.exponential(1 / 0.8, n)   # sosiri, λ = 0,8
S = rng.exponential(1.0, n)       # servire, μ = 1

Wq = np.zeros(n)                  # așteptările
for i in range(1, n):
    Wq[i] = max(0, Wq[i-1] + S[i-1] - A[i])

print(Wq.mean())   # teoretic ρ/(μ − λ) = 4
```

- Clientul i așteaptă cât a așteptat și a fost servit i − 1, minus cât a venit mai târziu
- Dacă rezultatul e negativ: 0 (găsește serverul liber)
- Rezultat: Wq = **4,11** (teoretic 4)
- 19,7% dintre clienți nu așteaptă deloc (teoretic 1 − ρ = 20%)

Așteptarea clienților consecutivi (ρ = 0,8)



- Așteptările vin în **valuri**: congestie, apoi eliberare
- Clienții consecutivi au așteptări **corelate**
- Media 4, dar unii așteaptă de 5 ori mai mult
- Un singur „vârf” poate dura zeci de clienți

Variabilitatea creează așteptare (ρ = 0,8, s = 1)

Wq = 0

D/D/1: sosiri la fiecare 1,25 u.t., servire exact 1 u.t. – nimeni nu așteaptă

Wq ≈ 2,0

M/D/1: sosiri aleatoare, servire constantă (teoretic 2)

Wq ≈ 3,9

M/M/1: sosiri și servire aleatoare (teoretic 4)

- Aceeași utilizare, aceeași capacitate: diferența o face **doar variabilitatea**
- Reducerea variabilității (loturi egale, cereri uniforme) scade așteptarea

Reguli practice din prima oră

- Utilizarea mare nu e un succes în sine: **latența** e ce simt utilizatorii
- Măsurați λ, s și W; verificați cu **Little** că numerele sunt consistente
- Găsiți **gâtul de sticlă** (Dmax) înainte de a cumpăra hardware
- Media ascunde vârfurile: raportați **p95, p99**
- Variabilitatea costă: uniformizați cererile unde se poate

✓ **Verificare de bun-simț**

Dacă $L \neq \lambda \cdot W$ în măsurători, ori contorizați greșit, ori sistemul nu e în regim staționar.

Cum analizăm un sistem cu cozi

Rețeta pe care o vom folosi toată săptămâna:

1 — Resursele

ce stații există și câte sunt (c)

2 — Măsurăm

λ, timpii de servire s (media și variabilitatea), vizitele

3 — Calcule rapide

ρ, gâtul de sticlă Dmax, Little

4 — Model

M/M/1, M/M/c, M/G/1 sau rețea (ora 2–3)

5 — Verificare

comparăm cu măsurători; la nevoie **simulăm** (S5)

Verificare

? Ce înseamnă **M/D/3/20** în notația Kendall?

A 3 sosiri, 20 de servere

B sosiri Poisson, servire constantă, 3 servere, cel mult 20 de clienți în sistem

C servire exponențială, sosiri constante, 3 cozi

D 20 de servere, fiecare cu 3 locuri

Verificare

? La un server sosesc în medie **30 de cereri/min**, iar servirea durează în medie **1,5 s**. Care este gradul de utilizare ρ?

A 0,2

B 0,5

C 0,75

D 1,5

Verificare

? Într-o cafenea sunt în medie **6** clienți, iar rata de sosire este **30 de clienți/oră**. Cât stă un client, în medie?

A 5 minute

B 12 minute

C 0,2 minute

D 180 de minute

Rezumatul orei 1

- ✓ Sistem de servire: clienți, stații, șir de așteptare, disciplină; **notația Kendall** A/B/c/K
- ✓ Sosiri **Poisson** (timpi exponențiali); indici: λ, μ, **ρ = λ/(cμ)**, X, W, Wq, L, Lq
- ✓ Stabilitate: **ρ < 1**; stabil nu înseamnă rapid
- ✓ **Legea lui Little: L = λW** – pentru orice sistem; legile operaționale, gâtul de sticlă
- ✓ Recurența lui Lindley simulează o coadă; **variabilitatea** creează așteptare

Ora 2

Modele clasice de cozi

1. M/M/1 ca lanț Markov: formule și curba latenței
2. M/M/c: mai multe servere, o coadă comună (Erlang C)
3. Capacitate finită: M/M/1/K și Erlang B
4. M/G/1: prețul variabilității (Pollaczek–Khinchine)

Unde am rămas

- ✓ Indici: λ, μ, **ρ = λ/(cμ)**, L, Lq, W, Wq; stabilitate: ρ < 1
- ✓ **Little**: L = λW, pentru orice sistem
- ✓ Variabilitatea creează așteptare: D/D/1 → 0, M/D/1 → 2, M/M/1 → 4
- ✓ Acum: **de unde** vin aceste valori

Procesul naștere–moarte

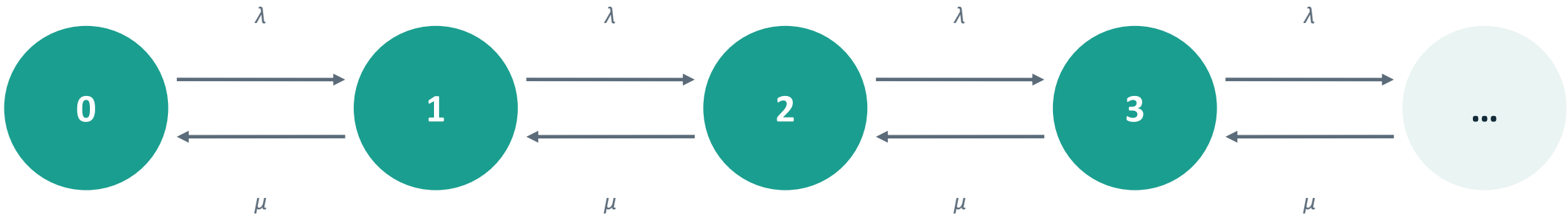
Proces naștere–moarte

Un lanț Markov în **timp continuu** în care starea k (numărul de clienți) crește cu 1 la o **sosire** (rata λ_k) și scade cu 1 la o **plecare** (rata μ_k).

Cambridge CSM; Holban, cap. 5

- Datorită lipsei de memorie, starea = numărul de clienți ajunge pentru a prezice viitorul
- Echilibru: fluxul k → k+1 = fluxul k+1 → k: $\boldsymbol{\pi_k \cdot \lambda_k = \pi_{k+1} \cdot \mu_{k+1}}$
- Aceeași idee ca $\pi_1 a = \pi_2 b$ de săptămâna trecută

M/M/1 ca lanț de stări



Din starea k: o sosire (rata **λ**) duce în k + 1, o plecare (rata **μ**) duce în k − 1. Starea 0 nu are plecări.

Distribuția numărului de clienți

$$\pi_k = (1 - \rho) \cdot \rho^k, \quad k = 0, 1, 2, \ldots$$

echilibru $\pi_k \cdot \lambda = \pi_{k+1} \cdot \mu \rightarrow \pi_{k+1} = \rho \cdot \pi_k \rightarrow \pi_k = \pi_0 \cdot \rho^k$

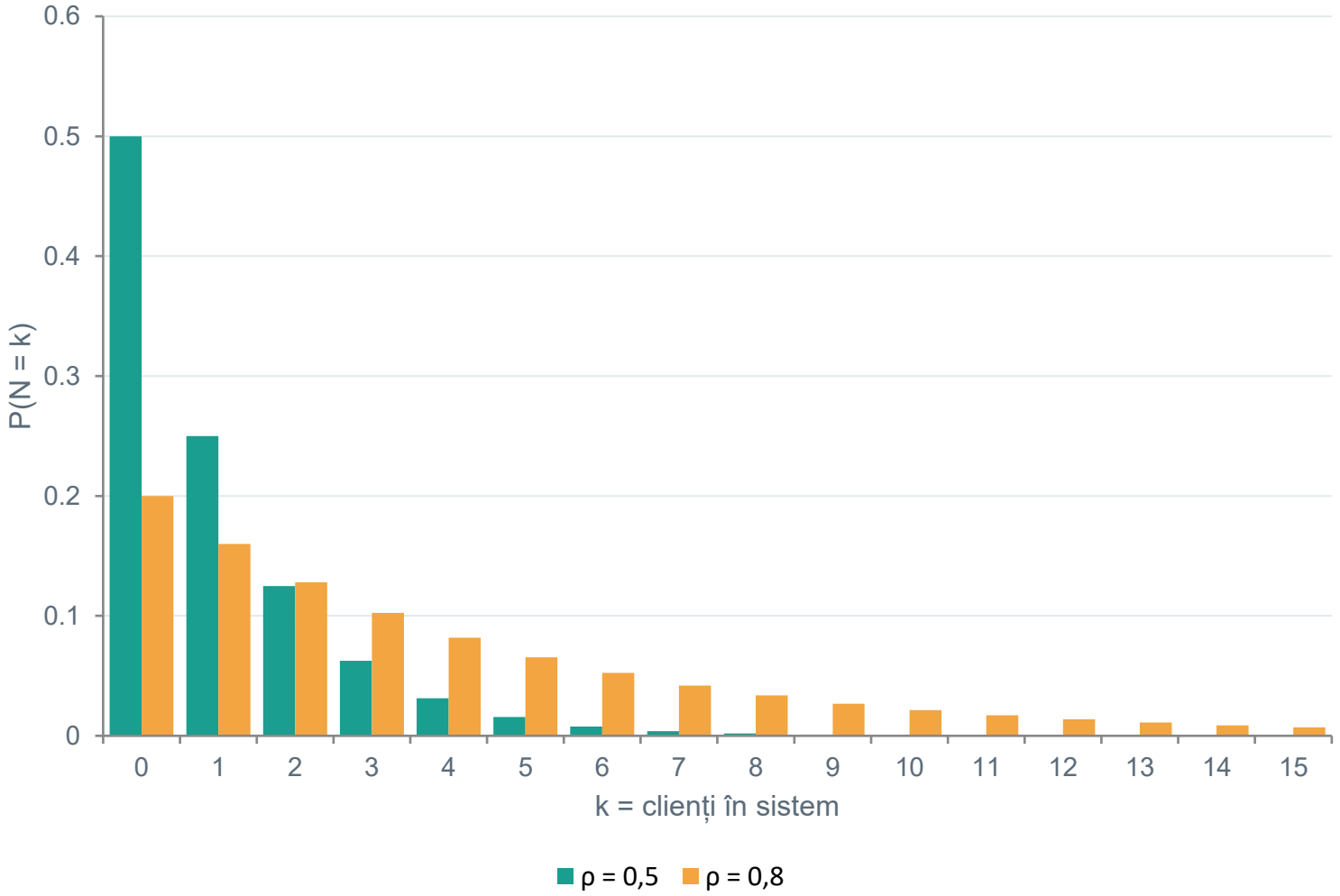
normare $\Sigma\, \pi_0 \rho^k = \pi_0 / (1 - \rho) = 1 \rightarrow$ **$\pi_0 = 1 - \rho$** (serverul e liber)

coada $P(N \geq n) = \rho^n$: probabilitatea de a găsi cel puțin n clienți

O distribuție **geometrică**: fiecare client în plus este de ρ ori mai puțin probabil.

Din echilibrul fluxurilor obținem că **fiecare stare are probabilitatea de rho ori probabilitatea stării anterioare**. Normând, adică impunând ca suma să fie unu, rezultă că **probabilitatea sistemului gol este unu minus rho**, deci serverul este ocupat exact o fracțiune rho din timp. Distribuția este geometrică, iar **probabilitatea de a găsi cel puțin n clienți este rho la puterea n**.

P(N = k) pentru ρ = 0,5 și ρ = 0,8



- ρ = 0,5: sistemul e gol jumătate din timp
- ρ = 0,8: gol doar 20% din timp; coada „lungă”
- P(N ≥ 10): 0,001 față de **0,107**
- O creștere mică a lui ρ lungește mult coada distribuției

Formulele M/M/1

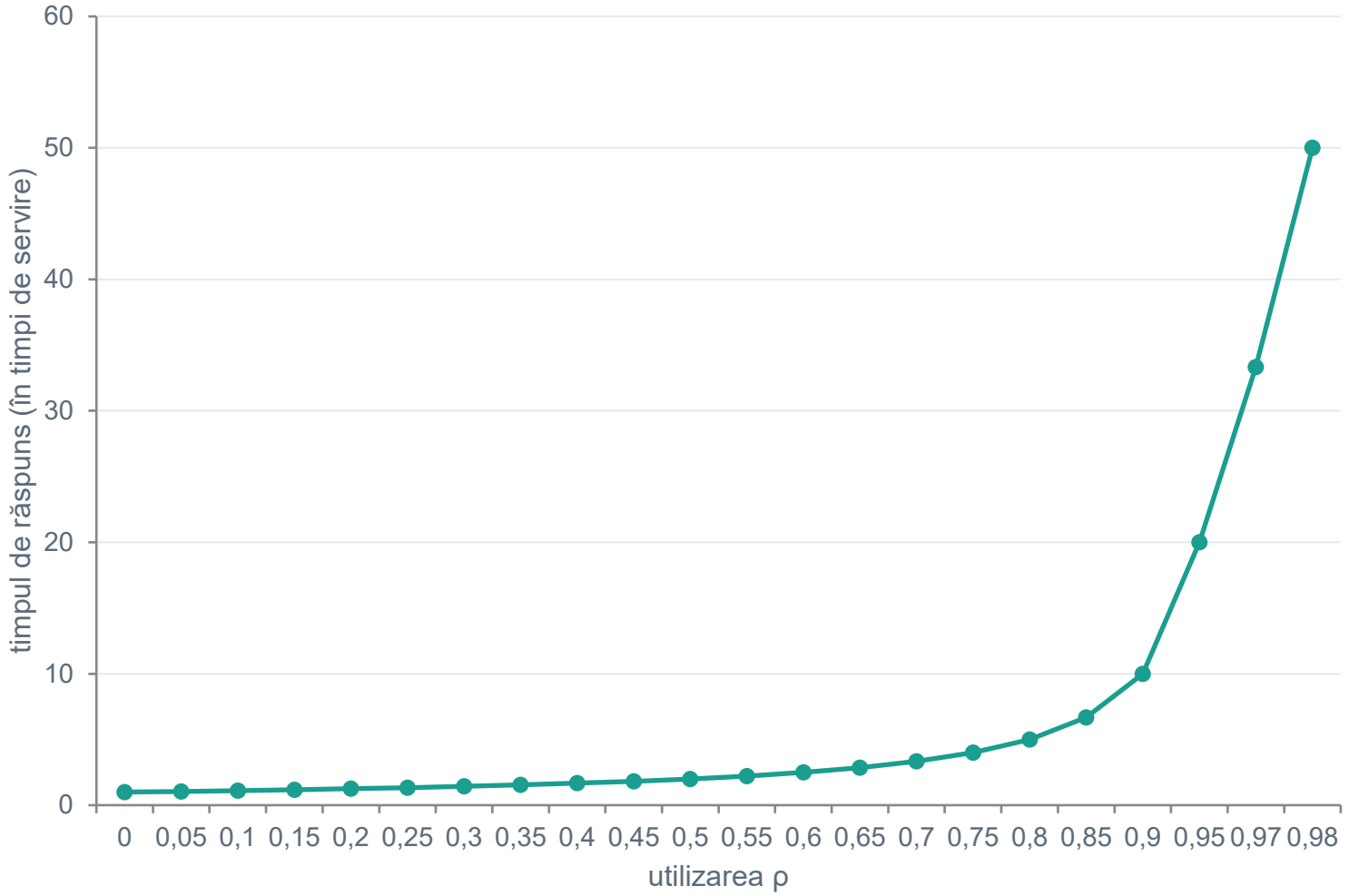
$$L = \rho / (1 - \rho) \qquad W = 1 / (\mu - \lambda)$$

Lq	$\rho^2 / (1 - \rho)$: clienți care așteaptă (L minus cei în servire, ρ)
Wq	$\rho / (\mu - \lambda)$: timpul de așteptare în coadă
Little	$W = L / \lambda = \rho / (\lambda(1 - \rho)) = 1 / (\mu - \lambda)$

Timpul de răspuns depinde de **diferența** μ − λ, adică de capacitatea rămasă liberă, nu de μ singur.

Din distribuție calculăm **numărul mediu de clienți**: rho pe unu minus rho. Cu legea lui Little obținem **timpul mediu de răspuns**: unu pe mu minus lambda. Formula are o interpretare foarte utilă: contează capacitatea rămasă liberă, diferența dintre rata de servire și rata de sosire. Când lambda se apropie de mu, numitorul tinde la zero și timpul de răspuns explodează.

Curba latenței: W / s = 1 / (1 – ρ)



- Aproape plată până la ρ ≈ 0,5
- La 0,8: **×5**; la 0,9: **×10**; la 0,95: **×20**
- Forma de „crosă de hochei”
- Ținta practică: ρ ≈ 0,6–0,7 pentru servicii interactive
- Aceasta este cea mai importantă figură a săptămânii. **Timpul de răspuns, măsurat în timpi de servire**, este unu pe unu minus rho. Curba este aproape plată până la 50 la sută, apoi urcă din ce în ce mai abrupt: de cinci ori la 80 la sută, de zece ori la 90, de douăzeci de ori la 95. De aceea **serviciile interactive sunt dimensionate pentru utilizări de aproximativ 60–70 la sută**.

Același +0,1 de încărcare, efecte foarte diferite (Cambridge)

ρ vechi → ρ nou	creșterea încărcării	W vechi → W nou (× 1/μ)	creșterea lui W
0,1 → 0,2	+100%	1,11 → 1,25	+13%
0,5 → 0,6	+20%	2 → 2,5	+25%
0,8 → 0,9	+12,5%	5 → 10	**+100%**

„Rularea la utilizare maximă îi mulțumește pe furnizori, dar nu și pe utilizatori.” (Gibbens, CSM)

Tabelul din cursul de la Cambridge arată efectul aceleiași creșteri a încărcării, de 0,1, în trei situații. La încărcare mică, dublarea traficului crește timpul de răspuns cu doar 13 la sută. La 0,8, o creștere de doar 12,5 la sută a traficului dublează timpul de răspuns. La încărcare mare, predicțiile devin foarte sensibile, iar un vârf mic de trafic poate produce o degradare mare.

$P(N \geq n) = \rho^n$

$0{,}5^3 = 0{,}125$

ρ = 0,5: cel puțin 3 clienți în **12,5%** din timp

$0{,}8^{10} \approx 0{,}11$

ρ = 0,8: cel puțin 10 clienți în **11%** din timp

$0{,}95^{20} \approx 0{,}36$

ρ = 0,95: cel puțin 20 de clienți în **36%** din timp

- Util pentru dimensionarea **bufferelor**: câte locuri ca P(N ≥ n) să fie mic?
- Exemplu Cambridge: λ = 10/s; ce μ asigură P(N ≥ 3) = 0,05? ρ = 0,05^(1/3) ≈ 0,37 → μ ≈ 27/s

Exemplu lucrat: frizeria (Cambridge)

Întrebare	Calcul	Rezultat
Utilizarea	$\rho = \lambda/\mu = 5/6$	0,833
Clienți în frizerie	$L = \rho/(1 - \rho)$	5
Clienți care așteaptă	$Lq = L - \rho$	4,17
Frizerul e liber	$\pi_o = 1 - \rho$	0,167 (1/6)
Timpul în frizerie / de așteptare	$W = 1/(\mu - \lambda)$; $Wq = W - 1/\mu$	60 min / 50 min
4 scaune: un client stă în picioare	$P(N \geq 5) = \rho^5$	0,402

λ = 5 clienți/oră (Poisson), în medie 10 min pe tunsoare → μ = 6/oră

Un exemplu lucrat din cursul de la Cambridge: o frizerie fără programare, unde sosesc cinci clienți pe oră, iar o tunsoare durează în medie zece minute. Utilizarea este cinci șesimi, deci în frizerie sunt în medie cinci clienți, dintre care peste patru așteaptă. Un client stă în medie o oră, din care cincizeci de minute așteaptă. Cu patru scaune în sala de așteptare, un client nou stă în picioare în 40 la sută din cazuri: frizerul chiar ar trebui să angajeze un ajutor.

Formula vs simulare

```
import numpy as np

for i, rho in enumerate([0.5, 0.8, 0.9, 0.95]):
    rng = np.random.default_rng(20 + i)
    n = 200_000
    A = rng.exponential(1 / rho, n)   # λ = ρ, μ = 1
    S = rng.exponential(1.0, n)
    Wq = np.zeros(n)
    for k in range(1, n):             # Lindley
        Wq[k] = max(0, Wq[k-1] + S[k-1] - A[k])
    W_sim = (Wq + S).mean()
    W_th = 1 / (1 - rho)              # 1/(μ − λ)
    print(rho, round(W_sim, 2), W_th)
```

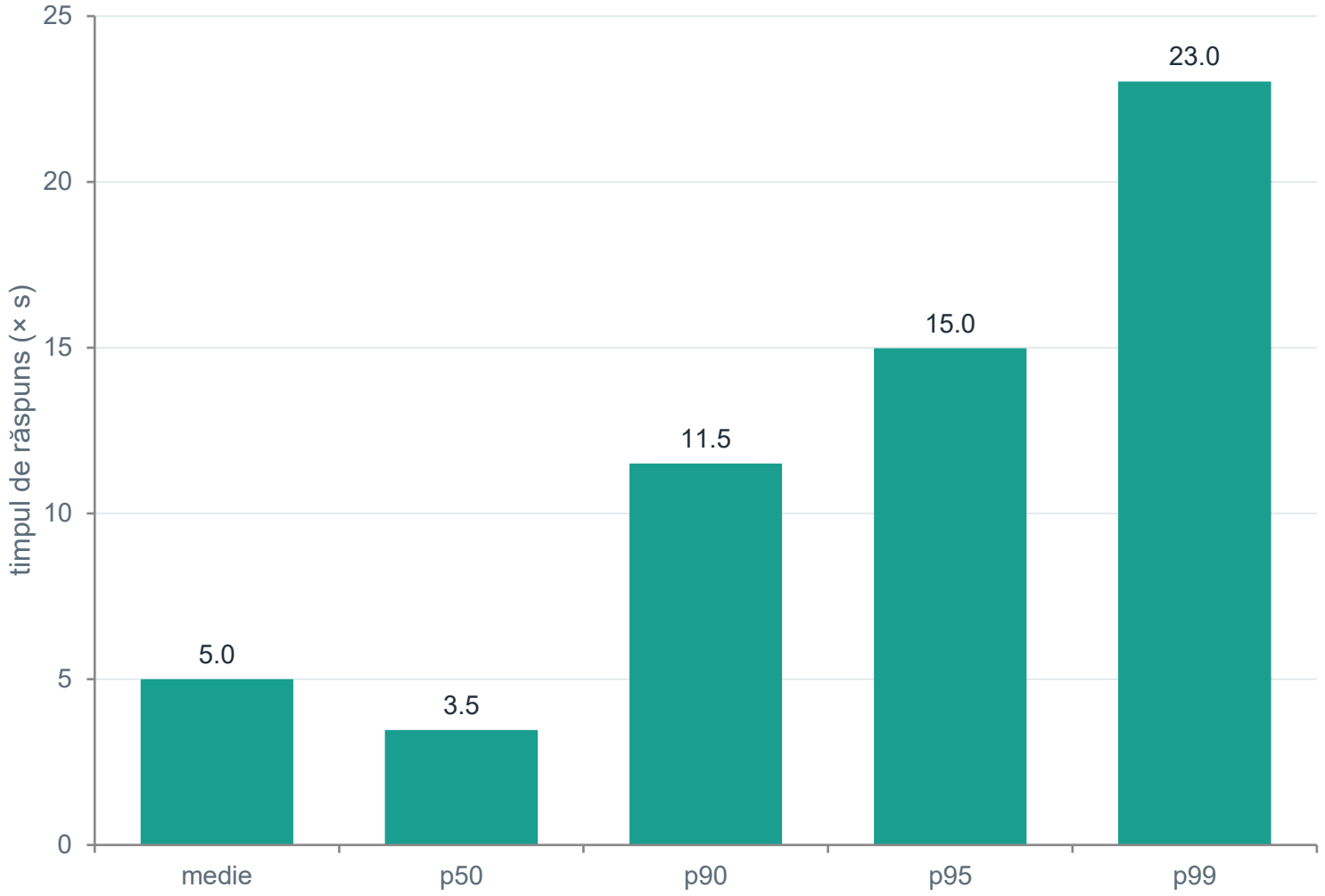
- Aceeași recurență Lindley din ora 1
- μ = 1, deci W e măsurat în timpi de servire
- 200 000 de clienți pentru fiecare ρ
- Rezultatele: pe slide-ul următor

Simulare vs teorie (M/M/1, 200 000 de clienți)

ρ	W simulat	W teoretic 1/(1 − ρ)	eroare relativă
0,5	2,01	2,00	0,5%
0,8	4,98	5,00	0,4%
0,9	10,41	10,00	4,1%
0,95	19,23	20,00	3,8%

Aproape de ρ = 1, simularea converge greu: clienții consecutivi sunt puternic corelați (S6)

Media nu spune tot: percentilele lui W (ρ = 0,8, s = 1)



- În M/M/1, W are distribuția **Exp(μ − λ)**
- Mediana (3,5) e **sub** medie (5)
- p99 = 23,0: de **4,6 ori** media
- SLA-urile se scriu pe percentile: „p99 < 300 ms”

Percentilele timpului de răspuns în M/M/1

$$W(q) = -\ln(1 - q) / (\mu - \lambda)$$

$p50$	ln 2 / (μ − λ) ≈ 0,69 · W
$p95$	ln 20 / (μ − λ) ≈ 3,0 · W
$p99$	ln 100 / (μ − λ) ≈ 4,6 · W

Aceeași transformare inversă ca în S2: X = −ln(U)/λ. Pentru alte modele, percentilele se obțin de obicei prin simulare.

Percentilele se obțin din funcția de repartiție exponențială, cu aceeași transformare inversă din săptămâna a doua. Mediana este 0,69 din medie, percentila 95 este de trei ori media, iar percentila 99 de 4,6 ori media. Aceste raporturi sunt valabile doar pentru M/M/1; pentru alte modele, percentilele se obțin de obicei prin simulare.

Verificare

? O coadă M/M/1 are **λ = 8 cereri/s** și **μ = 10 cereri/s**. Care este timpul mediu de răspuns W?

A 0,1 s

B 0,125 s

C 0,5 s

D 4 s

Verificare

? O coadă M/M/1 are ρ = **0,5**. Care este probabilitatea ca un client care sosește să găsească **cel puțin 3** clienți în sistem?

A 0,5

B 0,25

C 0,125

D 0,0625

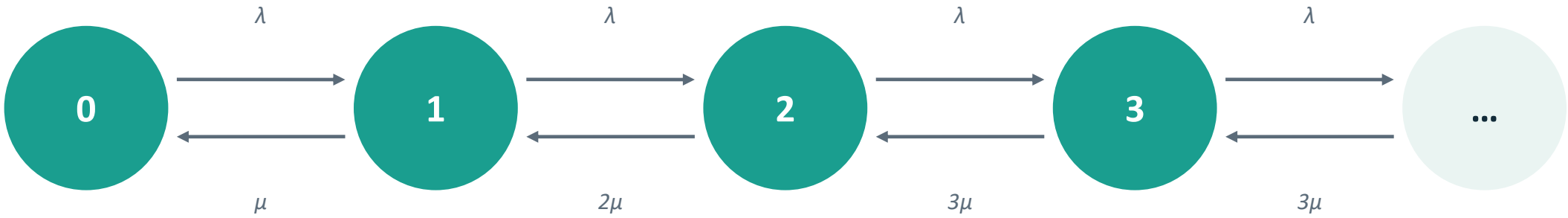
M/M/c: c servere, o coadă comună

Modelul M/M/c

Sosiri Poisson (λ), **c** servere identice, fiecare cu rata μ, și o **singură** coadă. Rata de plecare în starea k este **min(k, c)·μ**. Stabil dacă ρ = λ/(cμ) < 1.

- Exemple: call center, casele cu coadă unică, fire de execuție, grupuri de GPU
- a = λ/μ = „încărcarea oferită” în **erlangi** (câte servere ar fi ocupate în medie)
- Clientul așteaptă doar dacă **toate** serverele sunt ocupate

M/M/3 ca lanț de stări



Ratele de plecare cresc până la **3μ** (toate serverele ocupate), apoi rămân constante: de aici încolo apare coada.

Lanțul de stări pentru trei servere arată aproape la fel ca pentru unul, cu o singură diferență: rata de plecare crește cu numărul de clienți până când toate cele trei servere sunt ocupate. Din acel moment rămâne constantă, iar clienții în plus așteaptă în coadă.

Formula Erlang C

$$\textit{Probabilitatea de asteptare: } P(\text{așteptare}) = C(c, a),$$
$$\textit{Timpul mediu de asteptare: } Wq = C(c, a) / (c\mu - \lambda)$$

$C(c, a)$	probabilitatea ca un client să găsească toate cele c servere ocupate
calcul	prin Erlang B: C = B / (1 − ρ + ρB) (calcul numeric stabil – codul de la finalul orei)
W	Wq + 1/μ; L = λW (Little)

Aceeași structură ca la M/M/1: la numitor apare capacitatea liberă **cμ – λ**

Pentru M/M/c, probabilitatea ca un client să aștepte este dată de formula Erlang C, iar timpul mediu de așteptare este această probabilitate împărțită la capacitatea liberă.

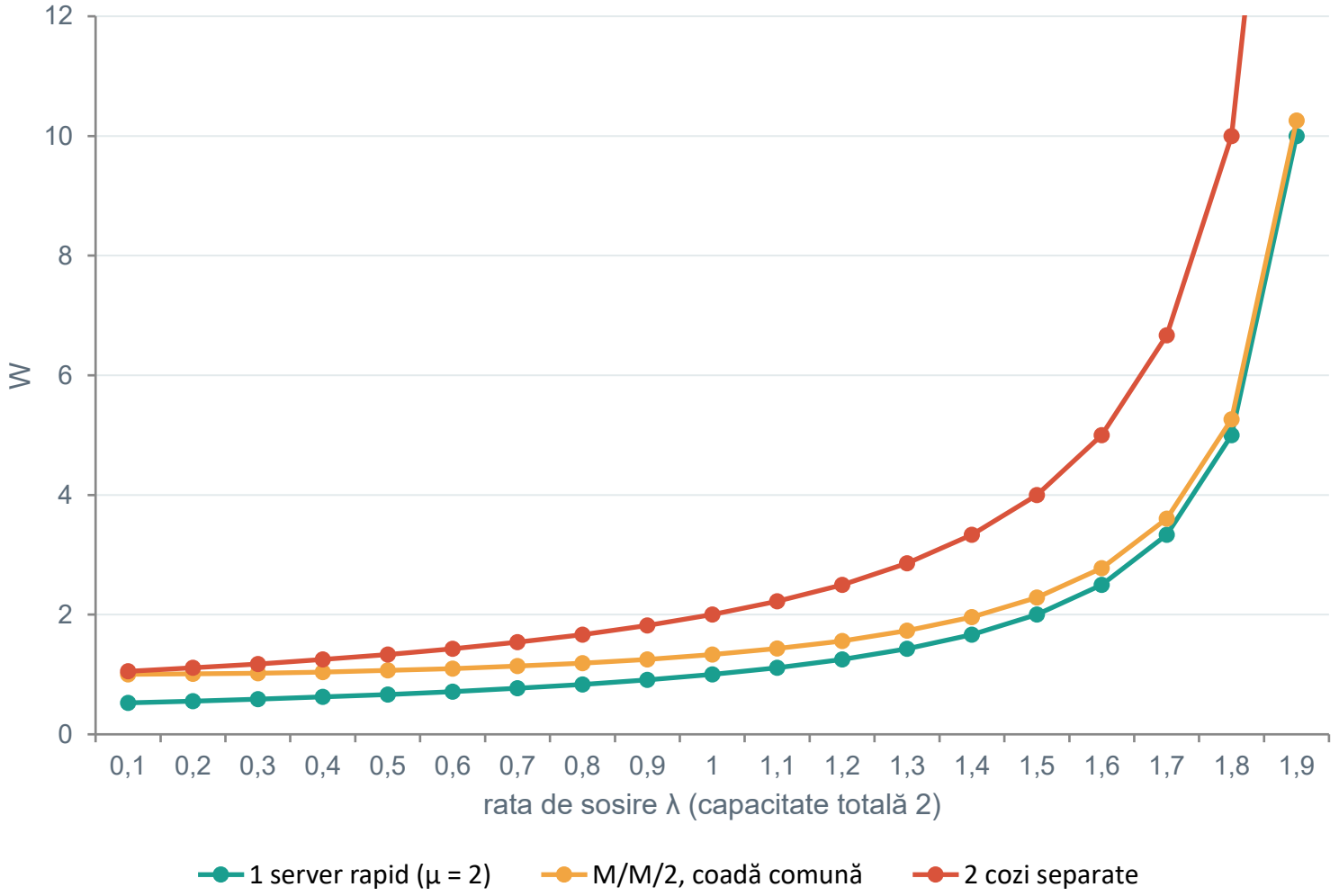
Un server rapid, două lente sau două cozi separate? (λ = 1,6)

Configurație (capacitate totală 2)	W	Wq	P(așteptare)
M/M/1, un server cu μ = 2	2,50	2,00	0,80
M/M/2, două servere cu μ = 1, coadă comună	2,78	1,78	0,71
2 × M/M/1, fiecare cu μ = 1, cozi separate	5,00	4,00	0,80

Aceeași capacitate totală și aceeași utilizare ρ = 0,8 în toate trei cazurile

O comparație clasică, cu aceeași capacitate totală și aceeași utilizare de 80 la sută. Un server rapid are timpul de răspuns 2,5. Două servere lente cu coadă comună au 2,78: puțin mai mult, pentru că servirea însăși durează dublu, dar așteptarea în coadă este mai mică. Două cozi separate au însă timpul de răspuns 5, dublu: un client poate aștepta la o coadă în timp ce serverul vecin stă liber.

Timpul de răspuns vs încărcare pentru cele trei configurații



- Coada comună ≈ serverul rapid la încărcare mare
- Cozile separate: mereu de ~2 ori mai lent
- La încărcare mică, serverul rapid câștigă (servire mai scurtă)
- Lecția: **o coadă comună** (pooling)

Lecțiile M/M/c

- **Pooling**: o coadă comună pentru mai multe servere reduce mult așteptarea
- De aceea: coadă unică la bancă, load balancer în fața serverelor, thread pool comun
- Economie de scară: 10 servere la $\rho = 0{,}9$ așteaptă mult mai puțin decât 1 server la $\rho = 0{,}9$
- Un server mai rapid scurtează și servirea: bun pentru latență la încărcare mică
- Multe servere: bun pentru **robustețe** (un server căzut nu oprește tot)

Exemplu

Casele de marcat cu coadă unică (tip „șerpuită”) sunt mai echitabile și mai rapide în medie decât cozile separate.

Verificare

? Două servere identice, ρ = 0,8. Ce configurație are timpul mediu de răspuns **mai mic**?

A două cozi separate, clienții aleg la întâmplare

B o coadă comună pentru ambele servere

C sunt identice, pentru că ρ e același

D depinde doar de disciplina FIFO

Capacitate finită: M/M/1/K

Modelul M/M/1/K

Un server și loc pentru cel mult **K** clienți în sistem (inclusiv cel în servire). Un client care găsește sistemul plin este **refuzat** (pierdut). Distribuția staționară există pentru orice ρ.

- Exemple: bufferul unui router (pachete aruncate), o coadă de mesaje limitată, HTTP 429
- Trade-off: K mare → mai puține pierderi, dar așteptări mai lungi
- Rata efectivă de sosire: $\lambda' = \lambda \cdot (1 - \pi(K))$

Probabilitatea de blocare (ca sistemul sa fie plin)

$$\pi(K) = (1 - \rho)\,\rho^{K} / (1 - \rho^{K+1})$$

π_k	$\pi_0 \cdot \rho^k$ pentru $k \le K$ (ca la M/M/1, dar trunchiat)
$\pi(K)$	probabilitatea ca sistemul să fie plin = fracțiunea clienților refuzați (PASTA)
$\rho = 1$	caz special: toate stările la fel de probabile, $\pi(K) = 1/(K + 1)$

Throughput-ul real este $\lambda(1 - \pi(K)) < \lambda$; utilizarea reală este $1 - \pi_0$.

Exemplu Cambridge: $\lambda = 10$, $\mu = 12$, $K = 15$

Întrebare	Calcul	Rezultat
Fracțiunea clienților refuzați	P_{15} cu $\rho = 5/6$	0,0114 (≈ 1,1%)
Rata efectivă de sosire	$\lambda(1 - P_{15})$	9,89 clienți/s
Utilizarea serverului	$1 - \pi_0 = \lambda' / \mu$	0,824

Cu 15 locuri pierdem ~1% din clienți; utilizarea (0,824) e puțin sub ρ = 0,833

Exemplul din foaia de probleme de la Cambridge: rata de sosire zece, rata de servire douăsprezece și cincisprezece locuri. Aproximativ un procent dintre clienți sunt refuzați, deci rata efectivă de sosire este 9,89 pe secundă. Utilizarea reală, 0,824, este puțin sub rho, tocmai pentru că o parte din trafic se pierde. Cu mai puține locuri pierderile ar crește rapid, iar cu mai multe ar crește așteptarea.

Erlang B: sisteme fără coadă (M/M/c/c)

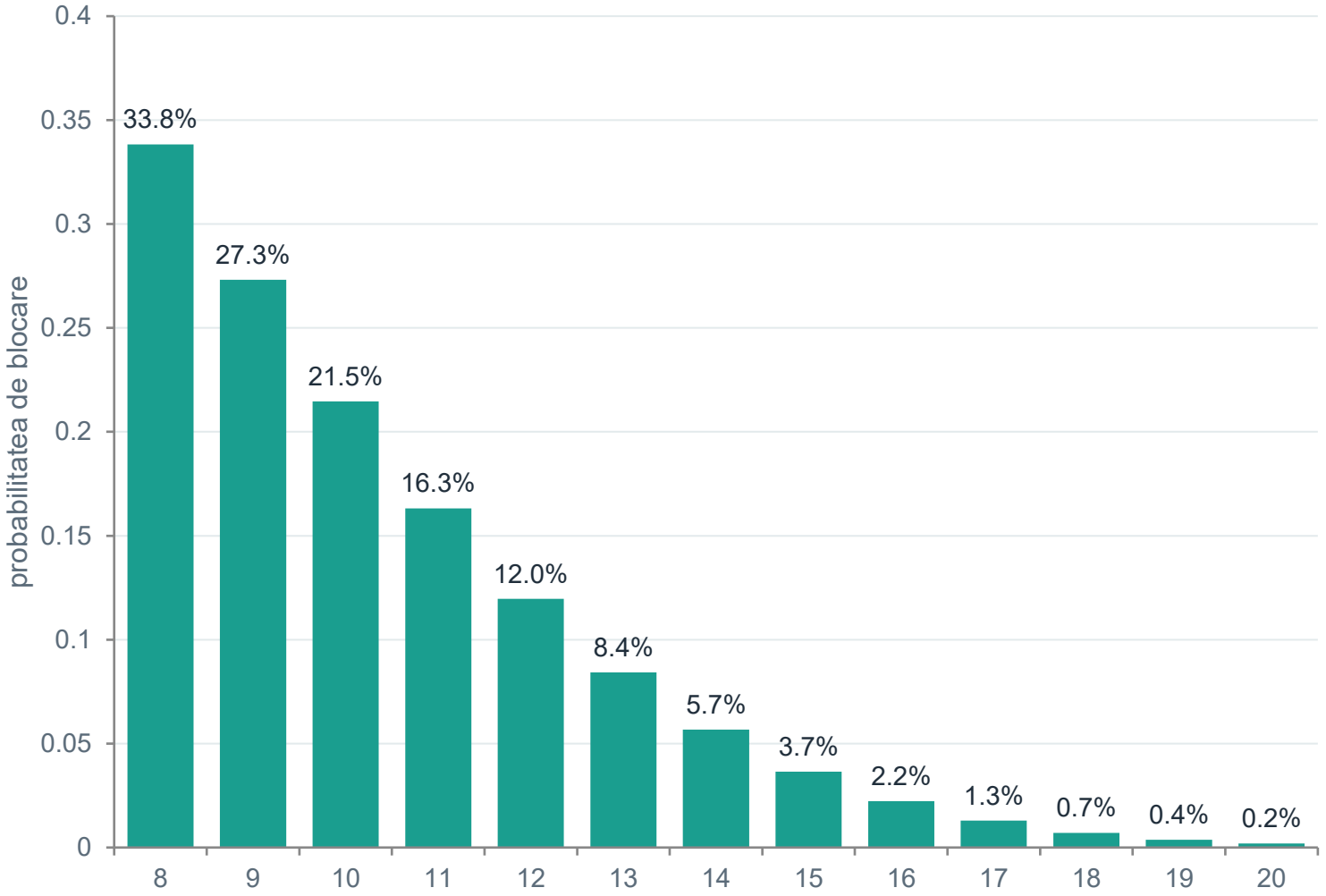
$$B(c, a) = (a^c / c!) / \Sigma_{k=0}{}^{c}\, a^k / k!$$

a	încărcarea oferită λ/μ (erlangi): de ex. 10 apeluri simultane în medie
B	probabilitatea ca **toate** cele c resurse să fie ocupate → cererea e pierdută
recurență	B(0) = 1; B(c) = a·B(c−1) / (c + a·B(c−1)) – stabilă numeric

Erlang (1917): câte linii telefonice? Azi: câte licențe, câte conexiuni simultane, câte instanțe fără coadă?

Când nu există deloc coadă, o cerere care găsește toate resursele ocupate este pierdută. Probabilitatea de pierdere este dată de formula Erlang B, dedusă de Erlang pentru liniile telefonice. Astăzi aceeași formulă dimensionează licențe software, conexiuni simultane sau instanțe fără coadă.

Câte resurse pentru a = 10 erlangi?



- c = 10: **21,5%** dintre cereri pierdute
- c = 15: 3,6%
- c = 18: **0,7%**
- Pentru 1% blocare: ~18 resurse, nu 10
- Pentru o încărcare oferită de zece erlangi, adică în medie zece cereri simultane, zece resurse nu sunt deloc suficiente: se pierd peste 21 la sută dintre cereri. Cu cincisprezece resurse pierderile scad sub 4 la sută, iar pentru mai puțin de un procent ne trebuie optsprezece. Din nou, variabilitatea cere capacitate suplimentară peste medie.

M/G/1: servire oarecare

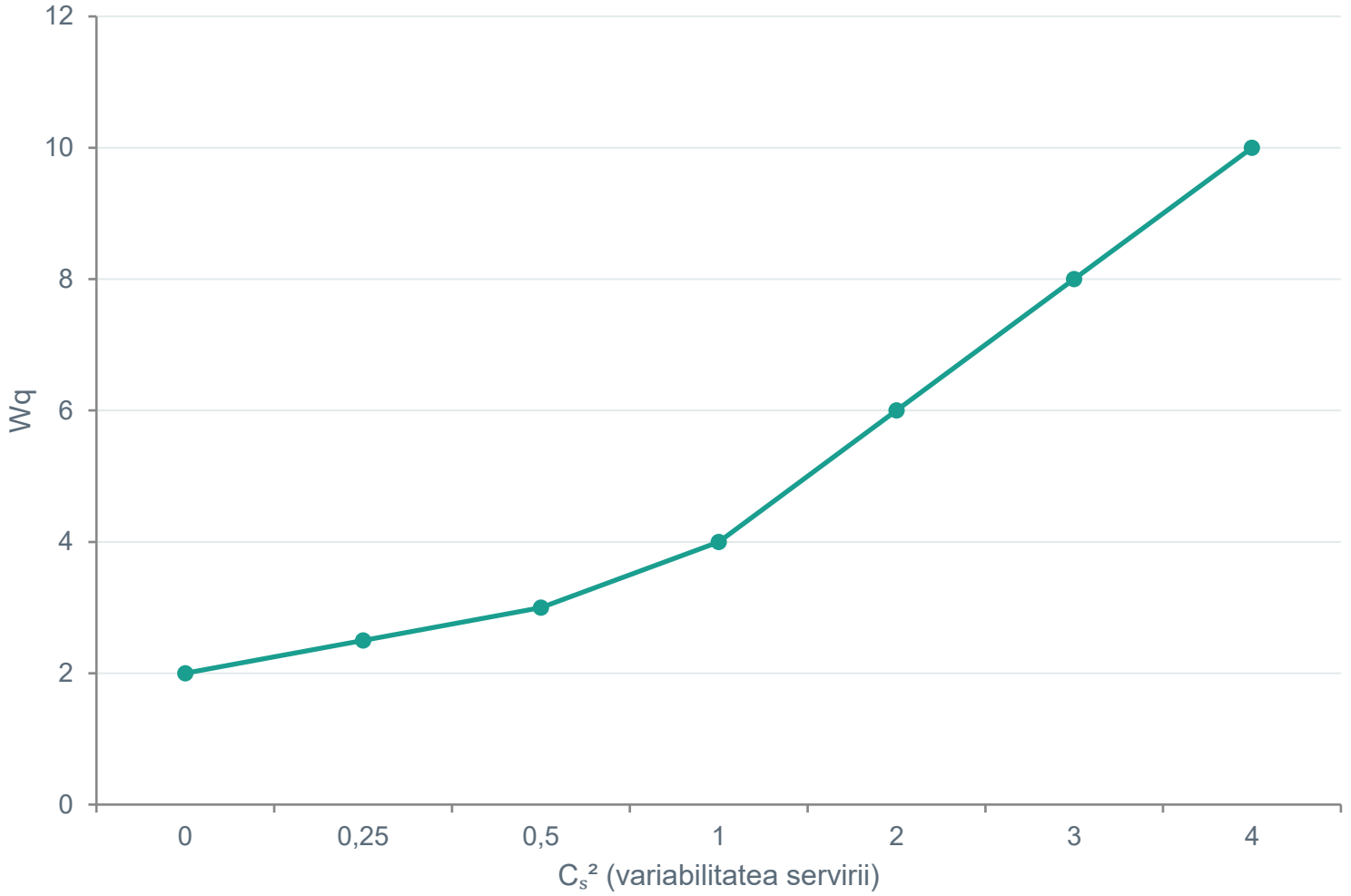
Formula Pollaczek–Khinchine

Pentru sosiri Poisson și servire cu **orice** distribuție (media s = 1/μ, coeficientul de variație C_s): $\mathbf{W_q = \rho\cdot(1 + C_s^2) / (2\mu(1 - \rho))}$.

Pollaczek (1930), Khinchine (1932); Cambridge CSM

- $C_s = 0$ (servire constantă, M/D/1): **jumătate** din așteptarea M/M/1
- $C_s = 1$ (exponențial): obținem exact M/M/1
- C_s mare (servire foarte variabilă, cozi grele): așteptări mult mai mari
- Când servirea nu este exponențială, starea nu mai este doar numărul de clienți, pentru că contează și cât a mai rămas din servirea curentă. Totuși, **pentru sosiri Poisson există o formulă exactă pentru așteptarea medie**, formula Pollaczek–Khinchine. Ea depinde doar de media servirii și de coeficientul ei de variație. Pentru servire constantă, așteptarea este jumătate din cea din M/M/1.

Așteptarea crește liniar cu variabilitatea (ρ = 0,8, s = 1)



- $C_s^2 = 0 \rightarrow Wq = 2$ (M/D/1)
- $C_s^2 = 1 \rightarrow Wq = 4$ (M/M/1)
- $C_s^2 = 4 \rightarrow Wq = 10$
- Timpii de servire foarte variabili costă cât o încărcare mult mai mare
- Graficul arată formula pentru utilizarea de 80 la sută. Așteptarea crește liniar cu pătratul coeficientului de variație al servirii: doi pentru servire constantă, patru pentru exponențială și zece când coeficientul de variație pătrat este patru.
- Pentru un server web, un amestec de cereri foarte scurte și foarte lungi produce exact un coeficient de variație mare.

Pollaczek–Khinchine verificat prin simulare (ρ = 0,8)

Servire (media 1)	C_s^2	Wq teoretic	Wq simulat
Constantă (M/D/1)	0	2	1,96
Exponențială (M/M/1)	1	4	3,89
Hiperexponențială ($M/H_2/1$)	4,0	10	9,58

400 000 de clienți; servirea H_2 = amestec de cereri scurte și lungi

G/G/1: aproximarea lui Kingman („VUT”)

$$Wq \approx V \cdot U \cdot T = ((C_a{}^2 + C_s{}^2)/2) \cdot (\rho/(1-\rho)) \cdot s$$

V	variabilitatea sosirilor (C_a) și a servirii (C_s)
U	utilizarea: factorul $\rho/(1-\rho)$, care explodează la $\rho \to 1$
T	timpul de servire s

Exactă pentru M/M/1 și M/G/1; aproximativă pentru G/G/1, dar foarte bună la încărcare mare (Kingman, 1961).

Pentru sosiri și serviri oarecare există o aproximare foarte utilă, datorată lui Kingman, numită și ecuația VUT. Așteptarea este produsul a trei factori: variabilitatea, utilizarea și timpul de servire. Pentru sosiri Poisson, factorul de variabilitate devine unu plus Cs pătrat pe doi, și regăsim formula Pollaczek–Khinchine. Aproximarea este foarte bună la încărcare mare.

Trei pârghii pentru a reduce așteptarea

1 V – variabilitatea

Cereri uniforme, loturi egale, programări, netezirea rafalelor (rate limiting).

2 U – utilizarea

Mai multe servere, autoscaling, refuzarea traficului în exces.

3 T – timpul de servire

Cod mai rapid, cache, hardware mai bun, batching.

Formulele în Python

```
def erlang_b(a, c):           # probabilitatea de blocare
    B = 1.0
    for k in range(1, c + 1):
        B = a * B / (k + a * B)
    return B

def erlang_c(a, c):           # P(așteptare) în M/M/c
    B, rho = erlang_b(a, c), a / c
    return B / (1 - rho + rho * B)

def mmc(lam, mu, c):
    C = erlang_c(lam / mu, c)
    Wq = C / (c * mu - lam)
    return Wq, Wq + 1 / mu

print(mmc(1.6, 1.0, 2))   # (1.778, 2.778)
```

- Recurența Erlang B evită factorialele mari
- Erlang C se obține din Erlang B
- mmc(λ, μ, 1) = formulele M/M/1
- Folosim aceste funcții în ora 3 pentru dimensionare

Formule pe scurt

Model	Distribuție / probabilitate	Timpi
M/M/1	$\pi_k = (1-\rho)\rho^k$; $P(N \geq n) = \rho^n$	$W = 1/(\mu-\lambda)$; $Wq = \rho/(\mu-\lambda)$
M/M/c	P(așteptare) = C(c, a)	$Wq = C/(c\mu-\lambda)$
M/M/1/K	$\pi(K) = (1-\rho)\rho^K/(1-\rho^{K+1})$	$\lambda' = \lambda(1-\pi(K))$
M/M/c/c	B(c, a) (Erlang B)	fără coadă
M/G/1	–	$Wq = \rho(1+C_s^2)/(2\mu(1-\rho))$
Orice sistem	$L = \lambda W$; $U = X \cdot S$	$X \leq 1/Dmax$

$\rho = \lambda/(c\mu)$, $a = \lambda/\mu$; toate cer $\rho < 1$ (cu excepția sistemelor cu pierdere)

Când formulele nu mai ajung

- Sosiri care **nu** sunt Poisson (rafale, zi/noapte) și serviri cu cozi grele
- Priorități, preempțiune, abandon (clienți care pleacă din coadă)
- Rețele cu **blocare**, resurse partajate, sincronizări (locks)
- Ne interesează **percentilele** sau regimul tranzitoriu, nu doar media
- → **simulare cu evenimente discrete** (săptămâna 5), validată pe cazurile cu formulă

Strategie

Formula = verificare rapidă și intuiție; simularea = răspunsul pentru sistemul real.

Verificare

? La aceeași utilizare ρ = 0,8 și același timp mediu de servire, cum se compară așteptarea medie **M/D/1** (servire constantă) cu **M/M/1**?

A este egală

B este **jumătate**

C este dublă

D este zero

Rezumatul orei 2

- ✓ M/M/1 = proces naștere–moarte: $\boldsymbol{\pi_k = (1 - \rho)\rho^k}$, $P(N \geq n) = \rho^n$
- ✓ **$W = 1/(\mu - \lambda)$**: curba latenței explodează la $\rho \to 1$; percentilele: $p99 \approx 4{,}6 \cdot W$
- ✓ M/M/c (Erlang C): **coada comună** (pooling) reduce mult așteptarea
- ✓ Capacitate finită: M/M/1/K (blocare), M/M/c/c (Erlang B)
- ✓ M/G/1: $Wq \propto (1 + C_s^2)$; Kingman: **$Wq \approx V \cdot U \cdot T$**

Ora 3

Rețele de cozi și sisteme reale

1. Rețele de cozi: ecuațiile de trafic, teorema lui Jackson
2. O aplicație web: unde se pierde timpul?
3. Dimensionare, autoscaling, latența de coadă (tail latency)
4. Servirea modelelor LLM ca problemă de cozi

Unde am rămas

- ✓ M/M/1: **$W = 1/(\mu - \lambda)$**; latența explodează la $\rho \to 1$
- ✓ M/M/c: coada comună (pooling); Erlang C pentru P(așteptare)
- ✓ Capacitate finită: blocare (M/M/1/K, Erlang B)
- ✓ Acum: **rețele** de astfel de cozi și decizii de proiectare

Rețele de cozi

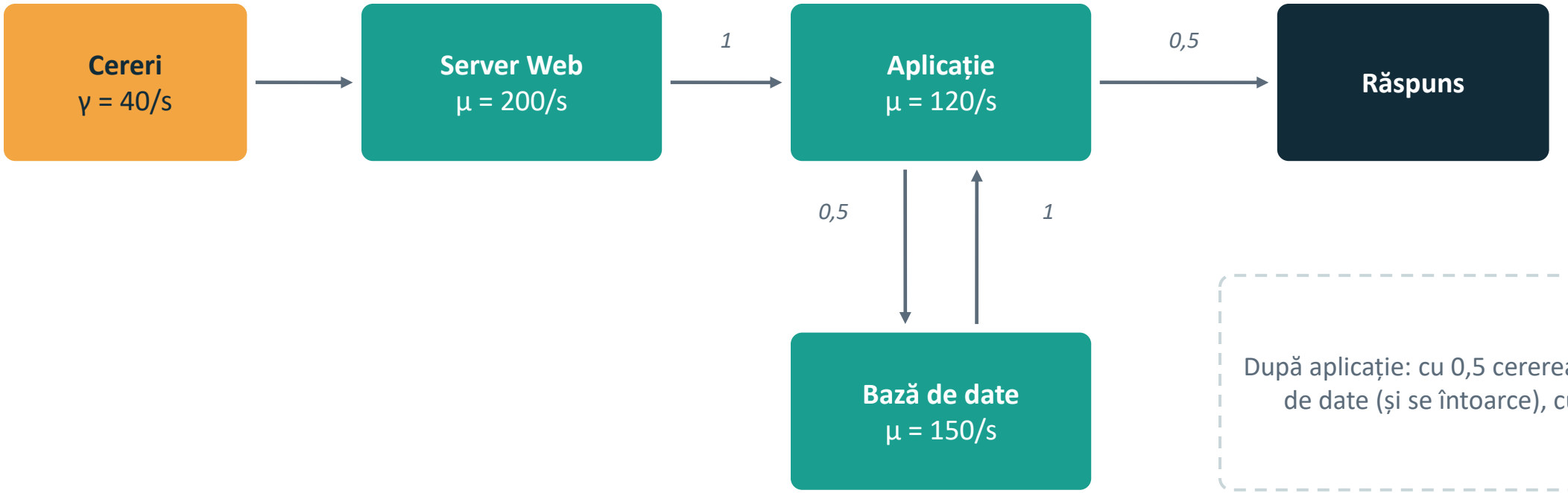
Rețea de cozi

Mai multe noduri (cozi cu servere), între care clienții se deplasează după **probabilități de rutare** q_{ij}. **Deschisă**: clienții intră din exterior și pleacă. **Închisă**: un număr fix de clienți circulă permanent.

Cambridge CSM; Harchol-Balter, 2013

- Deschisă: un serviciu web cu cereri din internet
- Închisă: N utilizatori interactivi (cer – așteaptă – gândesc – cer din nou), un pool fix de joburi
- Matricea de rutare Q seamănă cu o matrice de tranziție (S3), dar rândurile pot avea suma < 1

Exemplu: o aplicație web cu trei niveluri



După aplicație: cu 0,5 cererea merge la baza de date (și se întoarce), cu 0,5 pleacă

Ecuațiile de trafic

$$\lambda_i = \gamma_i + \Sigma_j \lambda_j \cdot q_{ji}$$

γ_i	sosirile din exterior la nodul i
$\lambda_j \cdot q_{ji}$	fluxul care vine de la nodul j către nodul i
exemplu	λ(web) = 40; λ(app) = 40 + λ(BD); λ(BD) = 0,5·λ(app) → **λ(app) = 80, λ(BD) = 40**

Un sistem liniar: $(I - Q^T)\lambda = \gamma$. Fiecare nod trebuie să aibă $\rho_i = \lambda_i/\mu_i < 1$.

Teorema lui Jackson (1957)

Rețea Jackson

Sosiri externe Poisson, servire exponențială, rutare probabilistică. În regim staționar, **fiecare nod se comportă ca o coadă M/M/1** (sau M/M/c_i) independentă, cu rata λ_i din ecuațiile de trafic.

- Numărul de clienți din noduri diferite este **independent**
- Timpul total al unei cereri = Σ (vizite la nodul i) × W_i
- Rezultat surprinzător: fluxul care intră într-un nod cu feedback nu e Poisson, dar formulele merg

Exemplul lucrat: γ = 40 de cereri/s

Nod	λ_i (/s)	ρ_i	vizite	W_i / vizită	L_i
Web	40	0,200	1	6,25 ms	0,250
Aplicație	80	0,667	2	25,00 ms	2,000
Bază de date	40	0,267	1	9,09 ms	0,364
Total					**2,614**

Timpul de răspuns: $T = \Sigma L_i / \gamma$ = 65,3 ms (Little pe toată rețeaua); simulare cu evenimente discrete: 65,4 ms

Tabelul arată calculul complet. Serverul web este ocupat 20 la sută din timp, aplicația 67 la sută, iar baza de date 27 la sută. La aplicație, o vizită durează în medie 25 de milisecunde, iar fiecare cerere o vizitează de două ori. Aplicând legea lui Little pe toată rețeaua, timpul de răspuns este numărul total de clienți împărțit la 40, adică 65,3 milisecunde; o simulare cu evenimente discrete, de tipul celor de săptămâna viitoare, dă 65,4.

Rețeaua în NumPy

```
import numpy as np
gamma = np.array([40.0, 0, 0])        # sosiri externe/s
Q = np.array([[0, 1.0, 0],            # web → app
              [0, 0,   0.5],     # app → BD 0,5; restul ies
              [0, 1.0, 0]])           # BD → app
mu = np.array([200.0, 120.0, 150.0])

lam = np.linalg.solve(np.eye(3) - Q.T, gamma)  # trafic
rho = lam / mu
L = rho / (1 - rho)                    # fiecare nod: M/M/1
T = L.sum() / gamma.sum()              # Little
print(lam, rho.round(3), round(1000 * T, 1), "ms")
```

- Ecuațiile de trafic: un sistem liniar
- Jackson: fiecare nod = M/M/1
- Little pe toată rețeaua dă timpul total
- Rezultat: [40, 80, 40], T = **65,3 ms**

Unde se pierde timpul?

50 ms

aplicația (2 vizite × 25 ms): 77% din timpul total

60/s

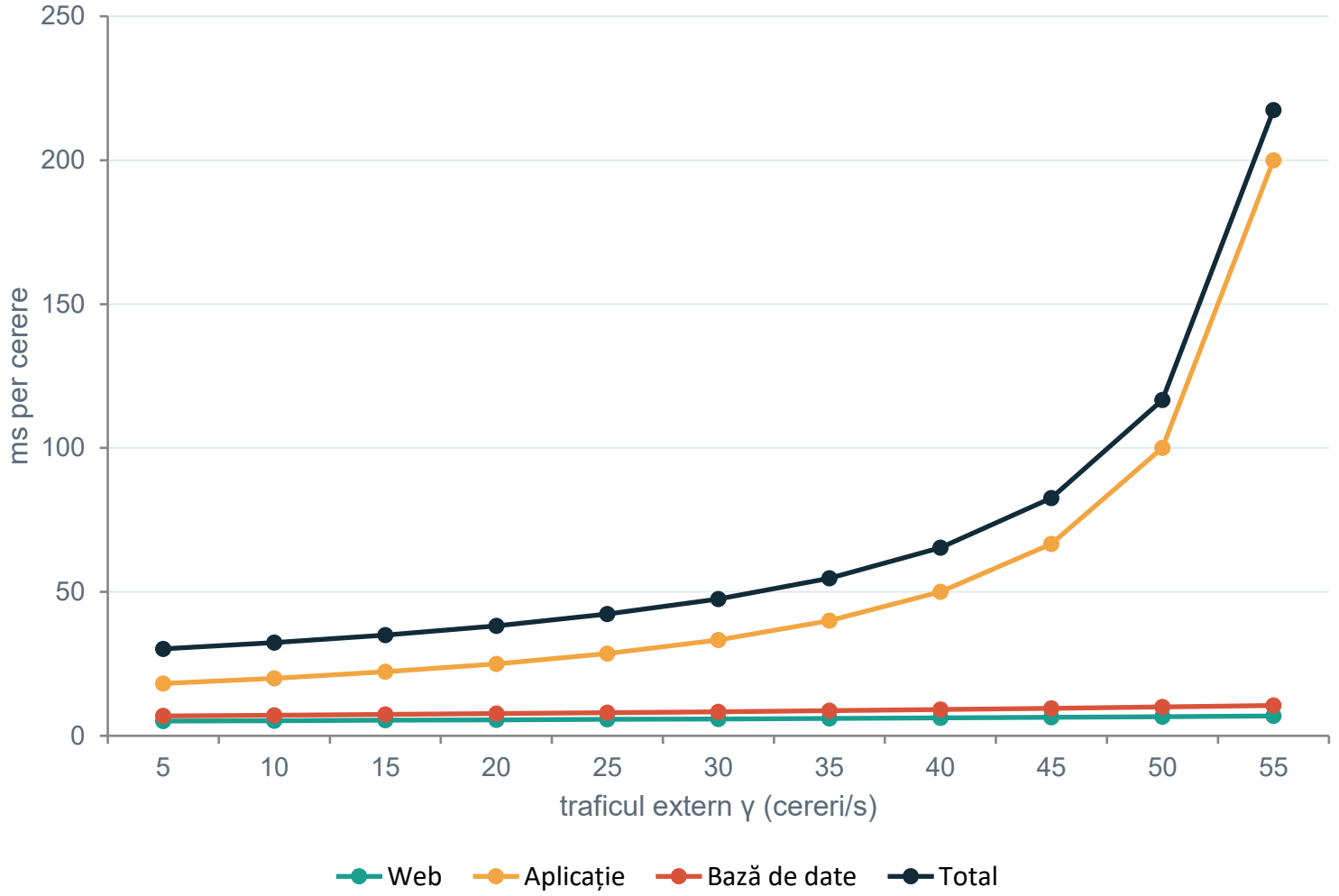
throughput maxim: aplicația se saturează când $2\gamma = 120$

15,3 ms

web + baza de date, împreună

- Gâtul de sticlă: nodul cu cererea de servire maximă $D_i = V_i \cdot S_i$ (**aplicația**: 2 × 8,3 ms)
- Optimizarea bazei de date ar câștiga cel mult câteva milisecunde
- Unde se pierde timpul? Aplicația consumă 50 din cele 65 de milisecunde, adică peste trei sferturi, pentru că este vizitată de două ori și este cel mai încărcat nod. Ea este și gâtul de sticlă: la 60 de cereri pe secundă ar fi saturată. Web-ul și baza de date împreună consumă doar 15 milisecunde, deci optimizarea bazei de date ar câștiga foarte puțin.

Timpul de răspuns pe noduri, când crește traficul



- Web și baza de date rămân aproape constante
- Aplicația explodează spre γ = 60/s
- Totalul urmează gâtul de sticlă
- Capacitatea se adaugă **unde** este gâtul de sticlă

Lecțiile rețelei

- Timpii se **adună** pe drumul cererii; vizitele repetate îi **înmulțesc**
- Gâtul de sticlă = nodul cu $\mathbf{D_i = V_i \cdot S_i}$ maxim; el limitează throughput-ul întregului sistem
- Optimizarea unui nod care nu e gât de sticlă aduce câștiguri mici
- Reducerea **vizitelor** (cache, interogări grupate) e adesea mai ieftină decât hardware-ul
- Verificați modelul cu măsurători; la nevoie, simulați (S5)

Exemplu

Un cache care elimină jumătate din apelurile app → BD scade λ(app) de la 80 la ≈ 53 de cereri/s (λ(app) = γ / (1 − 0,25)).

Efectul unui cache (app–BD: de la 0,5 la 0,25)

Nod	λ_i după (/s)	ρ_i după	ms/cerere înainte	ms/cerere după
Web	40,0	0,200	6,3	6,3
Aplicație	53,3	0,444	50,0	20,0
Bază de date	13,3	0,089	9,1	2,4
Total			**65,3**	**28,7**

Cache-ul nu face niciun server mai rapid: reduce doar numărul de vizite – și timpul de răspuns scade de peste 2 ori

Rețele deschise vs închise

Deschise

- clienți din exterior, cu rata γ
- throughput-ul = γ (dacă totul e stabil)
- întrebarea: ce latență la un trafic dat?
- soluție: ecuații de trafic + Jackson

vs

Închise

- N clienți circulă (utilizatori, joburi)
- throughput-ul **rezultă** din sistem
- întrebarea: ce throughput și latență pentru N utilizatori?
- soluție: legea timpului de răspuns, MVA (analiza valorii medii)

Utilizatori interactivi: legea timpului de răspuns

$$R = N / X - Z$$

N	numărul de utilizatori (sau clienți) din sistemul închis
X	throughput-ul măsurat (cereri/s)
Z	timpul de „gândire” dintre răspuns și cererea următoare

Exemplu: N = 100 de utilizatori, Z = 5 s, X = 18 cereri/s → R = 100/18 − 5 ≈ **0,56 s**. (Little aplicat întregului ciclu: N = X·(R + Z).)

Dimensionare și autoscaling

Câte servere ne trebuie?

1. Dimensionarea cu Erlang C
2. Regula rădăcinii pătrate
3. Autoscaling: de ce nu la 100%
4. Latența de coadă la scară mare (tail latency)

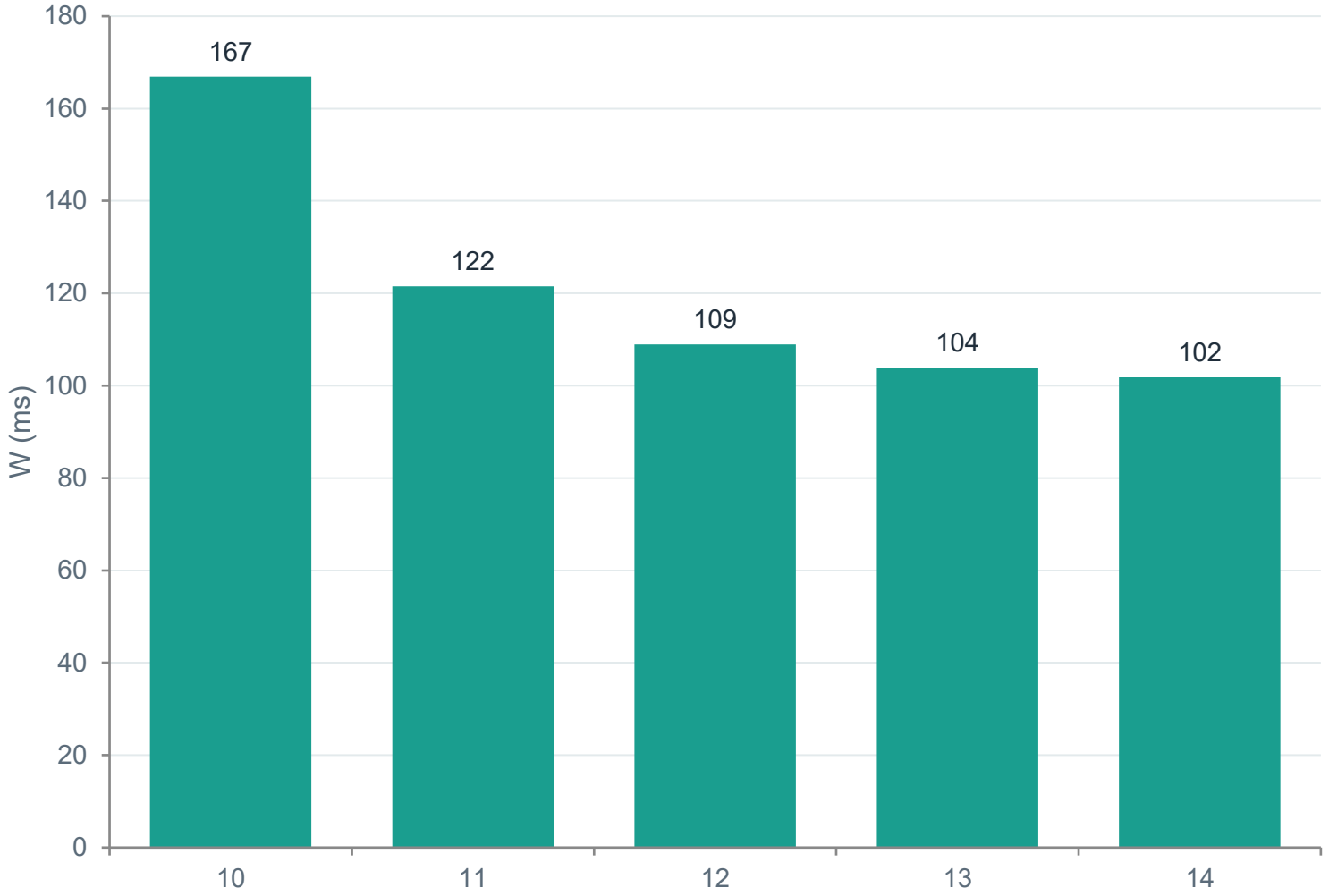
Dimensionare: λ = 90 de cereri/s, s = 100 ms (a = 9 erlangi)

Servere c	ρ	P(așteptare)	Wq (ms)	W (ms)
10	0,900	66,9%	66,9	166,9
11	0,818	43,0%	21,5	121,5
12	0,750	26,6%	8,9	108,9
13	0,692	15,8%	3,9	103,9
14	0,643	8,9%	1,8	101,8

Calculat cu funcția mmc (Erlang C) din ora 2; cu 9 servere sistemul ar fi instabil (ρ = 1)

Un exemplu: 90 de cereri pe secundă, fiecare cu o servire de o sută de milisecunde, deci o încărcare de nouă erlangi. Cu zece servere, sistemul este stabil, dar 67 la sută dintre cereri așteaptă, în medie 67 de milisecunde. Cu douăsprezece servere, doar 27 la sută așteaptă, iar timpul de răspuns scade la 109 milisecunde. Peste treisprezece-paisprezece servere câștigul devine foarte mic.

Timpul de răspuns vs numărul de servere



- Primul server peste minim aduce cel mai mult
- Apoi randamente descrescătoare
- Limita: timpul de servire (100 ms)
- Decizia: costul unui server vs costul latenței

Regula rădăcinii pătrate (square-root staffing)

$$c \approx a + \beta\sqrt{a}$$

servere = încărcarea medie + o **rezervă** proporțională cu √a

$$9 + 1 \cdot 3 = 12$$

a = 9, β = 1 → **12** servere (P(așteptare) ≈ 27%)

$$1/\sqrt{a}$$

rezerva **relativă** β√a / a scade cu mărimea sistemului: sistemele mari pot rula la utilizări mai mari

- β mai mare → mai puține așteptări (β = 2: 15 servere)
- Aceeași idee ca σ = √n din S3: fluctuațiile cresc doar ca radicalul

Autoscaling: de ce nu la 100%?

- În cloud, numărul de servere se ajustează automat după trafic, prin autoscaling
- Ținta tipică: utilizare de **60–70%** – rezervă pentru rafale și pentru curba latenței
- Pornirea unei instanțe durează (secunde – minute): în acest timp coada crește
- Metrici bune pentru scalare: lungimea cozii, latența p95, nu doar CPU-ul
- Scalare prea agresivă → oscilații; prea lentă → cozi lungi
- Serverless: „cold start” = un timp de servire mult mai mare pentru primele cereri

ρ **Legătura cu teoria**

La ρ = 0,65, W ≈ 3·s; la ρ = 0,9, W = 10·s. Rezerva de 30% cumpără o latență de 3 ori mai mică.

Latența de coadă la scară mare („The Tail at Scale”, 2013)

1%

un server răspunde lent (peste p99) într-o cerere din 100

100

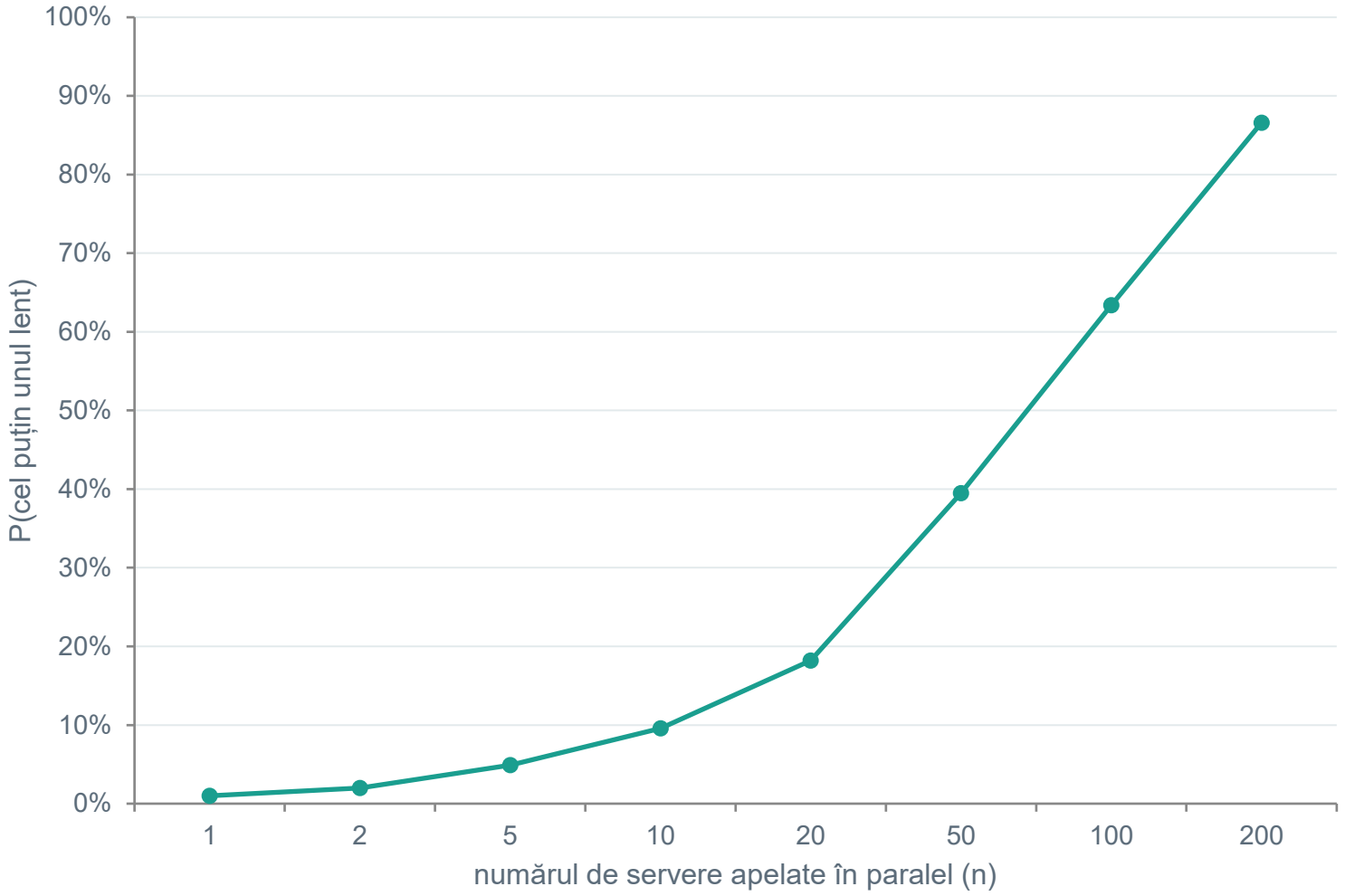
o cerere de utilizator apelează **în paralel** 100 de servere și le așteaptă pe toate

63%

dintre cererile utilizatorilor sunt lente: $1 - 0{,}99^{100}$

- Dean & Barroso (Google): la scară mare, percentilele rare devin cazul **obișnuit**
- Soluții: cereri duplicate („hedged”), timeout-uri, reducerea fan-out-ului

Probabilitatea ca o cerere cu fan-out n să fie lentă



- n = 10: 9,6%
- n = 50: 39,5%
- n = 200: **86,6%**
- Fiecare server are p99 „bun”, sistemul nu
- Graficul arată probabilitatea ca o cerere cu fan-out n să fie lentă, dacă fiecare server este lent în unu la sută din cazuri. La zece servere, aproape 10 la sută dintre cereri sunt lente; la două sute, 87 la sută.
- Fiecare server are o percentilă 99 bună, dar sistemul ca întreg nu. Este un argument puternic pentru a urmări percentilele, nu mediile.

Cum se reduce latența de coadă

- **Cereri duplicate** („hedged requests”): dacă un răspuns întârzie, trimitem cererea și altui server
- **Timeout-uri** și răspunsuri parțiale: mai bine un rezultat aproape complet la timp
- Mai puține servere apelate pe cerere (**fan-out** mai mic), agregare pe niveluri
- Cozi scurte și priorități pentru cererile interactive
- Monitorizare pe **p99 / p99,9**, nu pe medie

$ Cost

Cererile duplicate adaugă puțină încărcare (câteva procente), dar taie coada distribuției.

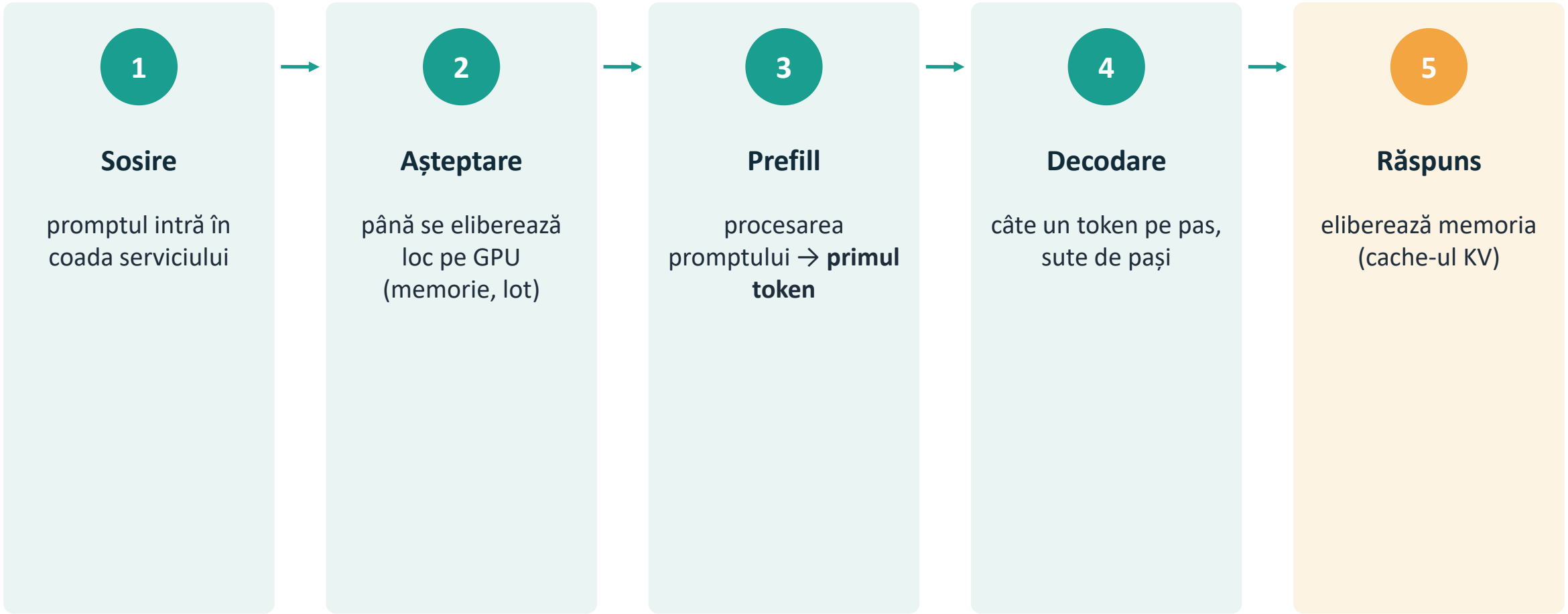
Servirea modelelor LLM

Inferența ca problemă de cozi

1. Ciclul unei cereri: prefill și decodare
2. Metrici: TTFT, TPOT, throughput
3. Batching static vs continuu
4. Memoria GPU ca resursă limitată

Ciclul unei cereri către un LLM

Timpul de servire nu e un singur număr: depinde de lungimea promptului și a răspunsului



Metricile servirii LLM

TTFT, TPOT, throughput

TTFT (time to first token) = așteptarea în coadă + prefill. **TPOT** (time per output token) = timpul dintre două tokenuri generate. **Throughput** = tokeni generați pe secundă de întregul sistem.

- Utilizatorul simte TTFT (cât „stă” până începe răspunsul) și TPOT (viteza textului)
- Operatorul urmărește throughput-ul pe GPU (cost pe token)
- Durata totală ≈ TTFT + (nr. de tokeni − 1) × TPOT

Little pentru un serviciu LLM

20/s

cereri sosite pe secundă

8 s

durata medie a unei generări (prefill + sute de tokeni)

160

conversații generate **simultan**, în medie → fiecare ocupă memorie GPU (cache KV)

- Memoria GPU limitează câte cereri încap simultan → un sistem cu capacitate finită (ca M/M/c/K)
- Răspunsuri mai lungi → W mai mare → mai multă memorie ocupată, la același trafic

Batching static vs continuu

Static (lot fix)

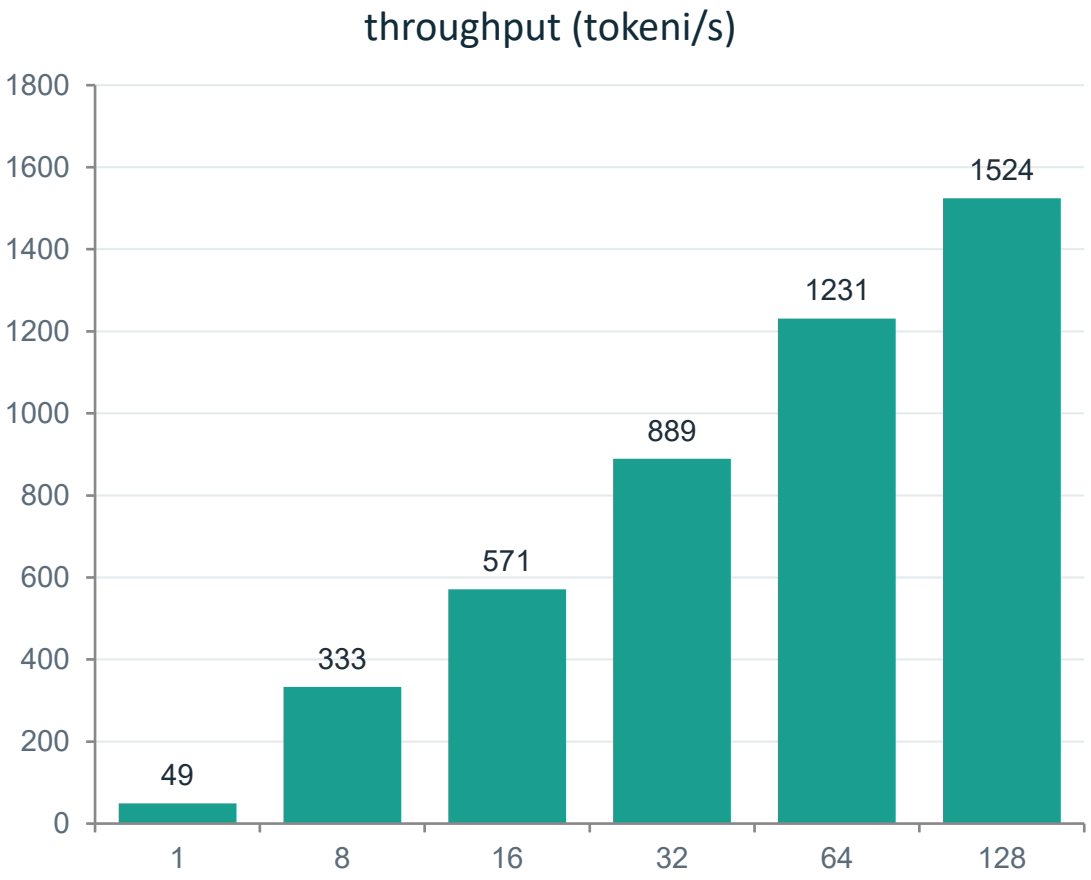
- se strâng B cereri, se procesează împreună până termină **toate**
- cererile scurte așteaptă după cele lungi
- cererile noi așteaptă terminarea lotului
- GPU folosit ineficient spre finalul lotului

vs

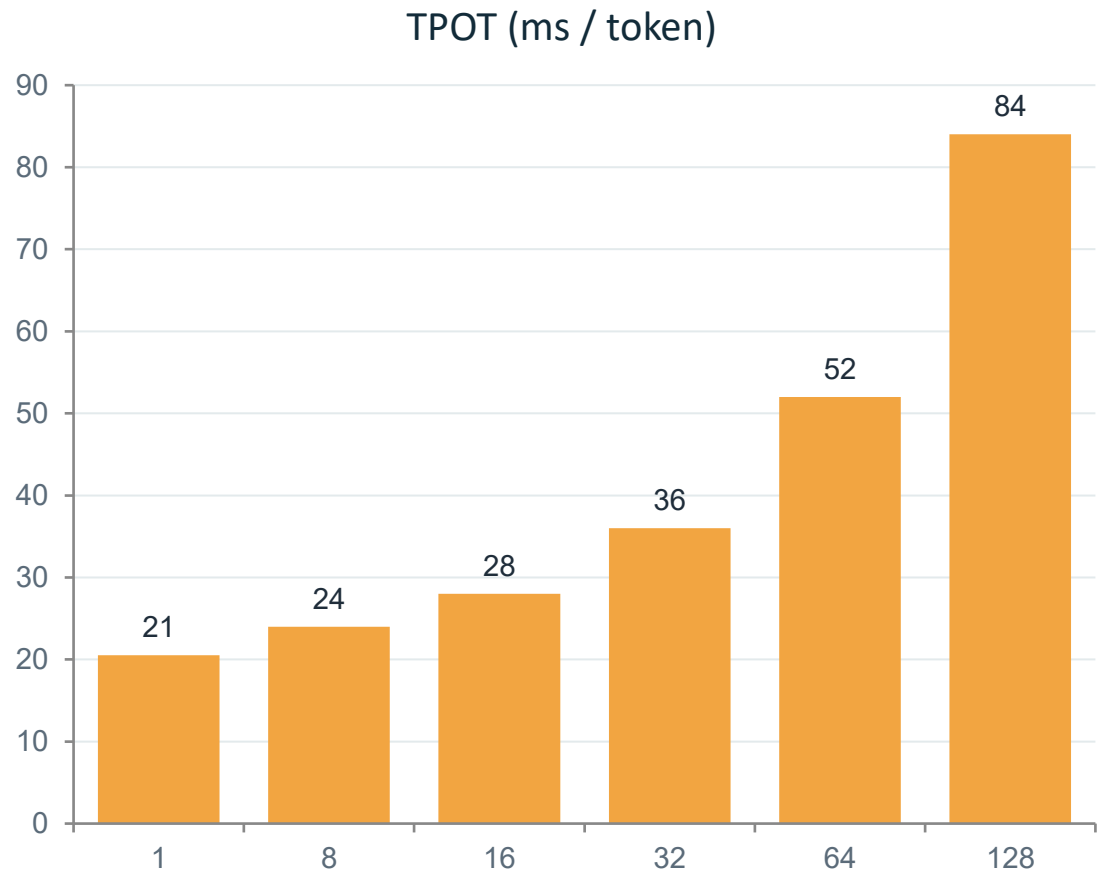
Continuu (Orca 2022; vLLM 2023)

- lotul se reface **la fiecare pas** de decodare
- o cerere terminată pleacă imediat, una nouă intră
- vLLM (PagedAttention): memoria KV gestionată pe pagini, ca memoria virtuală
- throughput mult mai mare la aceeași latență

Compromisul lotului (model ilustrativ: pas = 20 ms + 0,5 ms × B)



mai multe cereri per pas → throughput mai mare



dar fiecare pas durează mai mult

Controlul admiterii și prioritățile

- Când memoria sau coada e plină: cererile noi sunt **refuzate** (HTTP 429) sau puse în așteptare
- Limite de rată pe utilizator (token bucket): netezesc rafalele (pârghia V din VUT)
- Clase de prioritate: cereri interactive înaintea celor „batch”
- Ținte SLO pe percentile: de ex. TTFT p95 < 1 s, TPOT p95 < 100 ms
- Dimensionarea: câte GPU-uri pentru traficul de vârf și SLO? → Erlang + simulare

= **Aceleași idei**

M/M/c/K (refuz), priorități, Little, curba latenței – aplicate unei resurse de câteva zeci de mii de euro.

Alte cozi moderne în informatică

1 Cozi de mesaje

Kafka, RabbitMQ: producători și consumatori; întârzierea (lag) = coada.

2 Pool-uri de conexiuni

Conexiunile la BD sunt servere: M/M/c cu c = mărimea pool-ului.

3 Limitare de rată

Token bucket: permite rafale scurte, limitează media.

4 Rețele

Bufferele prea mari („bufferbloat”) cresc latența fără să crească throughput-ul.

5 Build-uri CI

Joburi care așteaptă mașini de build: cozi cu priorități.

6 Serverless

Instanțe pornite la cerere: M/M/∞ cu „cold start”.

De la formule la simulare (săptămâna 5)

- Formulele: rapide, intuitive, bune pentru **verificare** și dimensionare inițială
- Limitele lor: ipoteze Poisson/exponențiale, regim staționar, doar medii
- Simularea cu evenimente discrete: **orice** distribuție, priorități, blocări, percentile
- Validarea simulatorului: pe M/M/1, rezultatul trebuie să fie $1/(\mu - \lambda)$
- Săptămâna viitoare: scriem un simulator de la zero, apoi folosim SimPy

Cum lucrăm

Model analitic simplu → simulare detaliată → comparare cu măsurători.

Greșeli frecvente

- Dimensionare după **medie**: „în medie avem 9 erlangi, deci 9 servere” → sistem instabil
- Unități amestecate (cereri/min cu secunde) în λ, μ și Little
- Presupunerea Poisson/exponențial fără verificare pe date
- Raportarea doar a mediei; ignorarea **percentilelor** și a fan-out-ului
- Cozi separate unde s-ar putea folosi o coadă comună
- Optimizarea unui nod care nu e gâtul de sticlă

✓ **Verificare rapidă**

Calculați ρ pentru fiecare resursă. Dacă una e peste 0,8, acolo e problema.

Verificare

?

Într-o rețea, cererile intră cu γ = **30/s** și vizitează în medie de **3** ori serverul de aplicație, care are μ = **120/s**. Care este utilizarea lui?

A 0,25

B 0,5

C 0,75

D 1,33

Verificare

? O cerere de utilizator apelează în paralel **50** de servere; fiecare e lent în **1%** din cazuri, independent. Aproximativ ce fracțiune dintre cererile utilizatorilor sunt lente?

A 1%

B 5%

C 40%

D 50%

Verificare

? Traficul oferit este **a = 16 erlangi**. După regula rădăcinii pătrate cu **β = 1**, câte servere alegem?

A 16

B 17

C 20

D 32

Verificare

? Un serviciu LLM primește **10 cereri/s**, iar o generare durează în medie **6 s**. Câte generări rulează simultan, în medie?

A 0,6

B 1,67

C 16

D 60

Rezumatul săptămânii 4

- ✓ Kendall A/B/c/K; indici **$\rho = \lambda/(c\mu)$**, L, W; stabilitate $\rho < 1$; **$L = \lambda W$** (Little)
- ✓ Gâtul de sticlă: Dmax = max $V_i \cdot S_i$; $X \leq 1/Dmax$; variabilitatea creează așteptare
- ✓ M/M/1: **$W = 1/(\mu - \lambda)$**, $P(N \geq n) = \rho^n$; curba latenței; p99 ≈ 4,6·W
- ✓ M/M/c (Erlang C): coada comună; M/M/1/K, Erlang B: blocare
- ✓ M/G/1: $Wq \propto (1 + C_s^2)$; Kingman **V·U·T**
- ✓ Rețele Jackson: ecuații de trafic, timpi adunați; dimensionare $c \approx a + \beta\sqrt{a}$
- ✓ Tail at scale; servirea LLM: TTFT, TPOT, batching continuu, Little pentru memorie

Săptămâna viitoare

Simularea cu evenimente discrete

- Stare, eveniment, ceas simulat; time-stepping vs event-driven
- Un motor DES de la zero (heapq); M/M/1 simulat vs formula de azi
- SimPy: procese și resurse; intersecții, producție, urgențe

Cum simulăm o zi întreagă dintr-o bancă în câteva milisecunde, fără să așteptăm nicio secundă reală?

Bibliografie pentru săptămâna 4

- Ș. Holban – Tehnici de modelare și simulare, cap. 2 (graful activităților) și cap. 3 (sisteme cu șiruri de așteptare)
- R. J. Gibbens – Computer Systems Modelling, Cambridge – Little, M/M/1, M/M/c, M/M/1/K, M/G/1, rețele Jackson
- M. Harchol-Balter – Performance Modeling and Design of Computer Systems, Cambridge University Press, 2013
- L. Kleinrock – Queueing Systems, Volume 1: Theory, Wiley, 1975
- R. Jain – The Art of Computer Systems Performance Analysis, Wiley, 1991 (legile operaționale)
- J. D. C. Little – A Proof for the Queuing Formula L = λW, Operations Research, 1961
- J. Dean, L. A. Barroso – The Tail at Scale, Communications of the ACM, 2013
- W. Kwon et al. – Efficient Memory Management for LLM Serving with PagedAttention (vLLM), SOSP 2023
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5